LAB 3: Physical Design



1 Mục tiêu bài lab

Nắm được các khái niệm của Physical Design, phân tích với OpenRoad, tcl cơ bản. Làm được RTLtoGDS 2 con chip số phổ biến là cpu và TDC

2 Thí nghiệm 1

Ví dụ với cpu8bit:

Thiết kế RTL chi tiết đọc trong file README.md của dự án và xem code RTL trong thư mục nhé.

Liệt kê các file .odb (bản lưu trạng thái thiết kế chip qua từng bước), đây là file định dạng mà openroad chấp nhận để mở và phân tích. Tại mỗi bước mà openroad thực thi thành công, nó sẽ tạo file .odb tương ứng. Bước bị lỗi sẽ không tạo .odb, ở đây bước 40 bị lỗi, do đó sẽ không tạo .odb.

Code 2.1: Chọn thư mục chạy, lưu với biến $RUN, rồi liệt kê các file .odb được tạo.

```
RUN="$HOME/ctwsemi-data/ips/cpu8bit/runs/cpu8bit-20ns"
```

```
huyatieo@huyatieo-ThinkPad-X13-Gen-1:~$ RUN="$HOME/ctwsemi-data/ips/cpu8bit/runs/cpu8bit-20ns"

find "$RUN" -type f -name '*.odb' -printf '%P\n' \
  | sort -V \
  | nl -w2 -s'. '
 1. 14-openroad-floorplan/tt_um_dvxf_dj8.odb
 2. 17-odb-setpowerconnections/tt_um_dvxf_dj8.odb
 3. 19-openroad-cutrows/tt_um_dvxf_dj8.odb
 4. 20-openroad-tapendcapinsertion/tt_um_dvxf_dj8.odb
 5. 22-openroad-generatepdn/tt_um_dvxf_dj8.odb
 6. 28-odb-applydeftemplate/tt_um_dvxf_dj8.odb
 7. 29-openroad-globalplacement/tt_um_dvxf_dj8.odb
 8. 33-openroad-repairdesignpostgpl/tt_um_dvxf_dj8.odb
 9. 35-openroad-detailedplacement/tt_um_dvxf_dj8.odb
10. 36-openroad-cts/tt_um_dvxf_dj8.odb
11. 38-openroad-resizertimingpostcts/tt_um_dvxf_dj8.odb
```

```
find "$RUN" -type f -name '*.odb' -printf '%P\n' \
  | sort -V \
  | nl -w2 -s'. '
```

Hình 2.1: liệt kê các file .odb chứa bản lưu trạng thái chip qua từng bước

Chạy lệnh này để mở .odb của step tương ứng. Ví dụ với step29, global placement, lúc này các standard cell được phép đặt chồng lần lên nhau để tối ưu vị trí, đồng thời các physical cell như endcap và tap cell cũng đã được thêm vào. Chưa chèn các cell fix timing.

Trước khi chạy lệnh dưới đây ở terminal hãy khởi động openROAD GUI trên ctwsemi trước (vào layout -> chọn tên mục run -> mở OpenROAD GUI).

Code 2.2: Khởi chạy openROAD và hiển thị trạng thái chip (.odb) tại bước 29 global placement

```
RUN="$HOME/ctwsemi-data/ips/cpu8bit/runs/cpu8bit-20ns"
STEP="29-openroad-globalplacement"
ODB="$RUN/$STEP/tt_um_dvxf_dj8.odb"

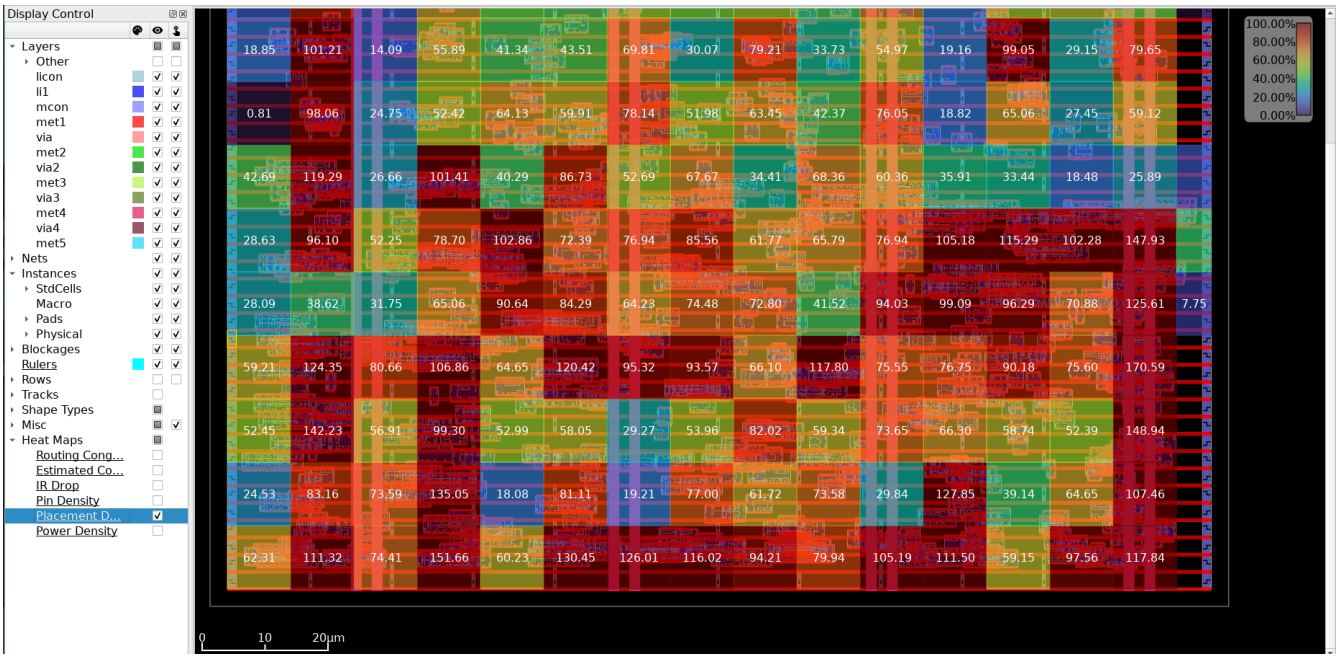
OR_CID="$(docker ps \
  --filter "ancestor=ghcr.io/librelane/librelane:3.0.14" \
  --format '{{.ID}}' | head -n 1)"

if [ ! -f "$ODB" ]; then
    echo "Không tìm thấy ODB: $ODB"
elif [ -z "$OR_CID" ]; then
    echo "Không tìm thấy container LibreLane GUI."
    docker ps
else
    echo "Container: $OR_CID"
    echo "ODB: $ODB"

    docker exec -d \
      -e DISPLAY="$DISPLAY" \
      "$OR_CID" \
      openroad -gui -db "$ODB"

    echo "Đã khởi chạy OpenROAD cho step $STEP"
fi
```

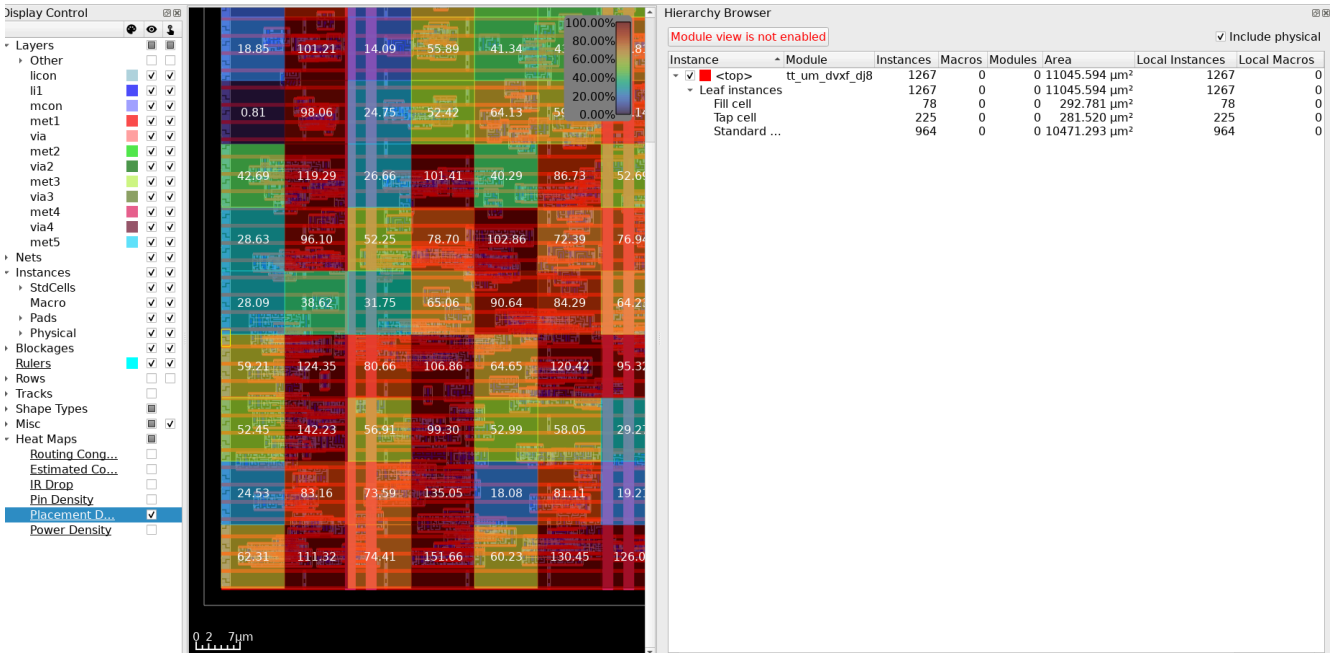
Kết quả **Heat Maps Placement Density** khi chạy step29:



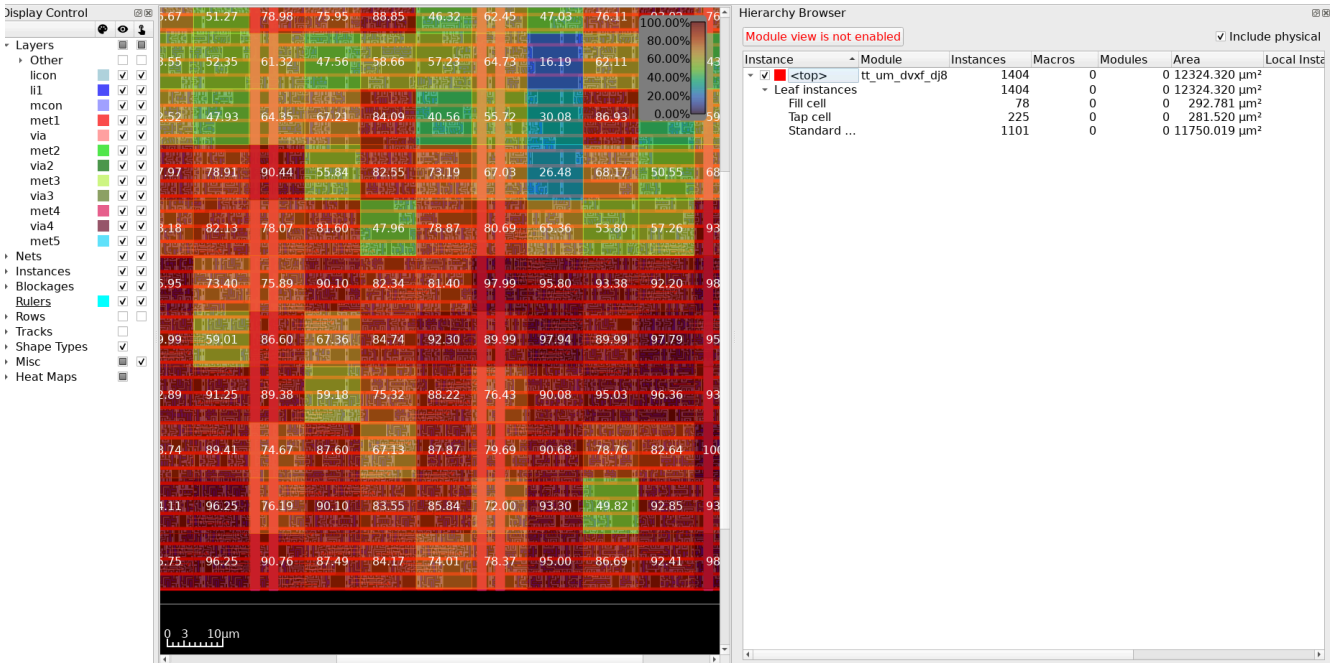
Hình 2.2: Bản đồ nhiệt của chip sau khi đặt các cell tại bước 29 global placement

Nhận xét: Placement của một vài khu vực vượt trên 100% vì các cell bị đặt chồng lấn lên nhau.

Kết quả của hierarchy browser tại step Global Placement. Nếu click chọn Include Physical (bao gồm cả fill cell và tap cell), tổng cell là 1267 (78 Fill Cell, 225 Tap Cell và 964 Standard Cell).



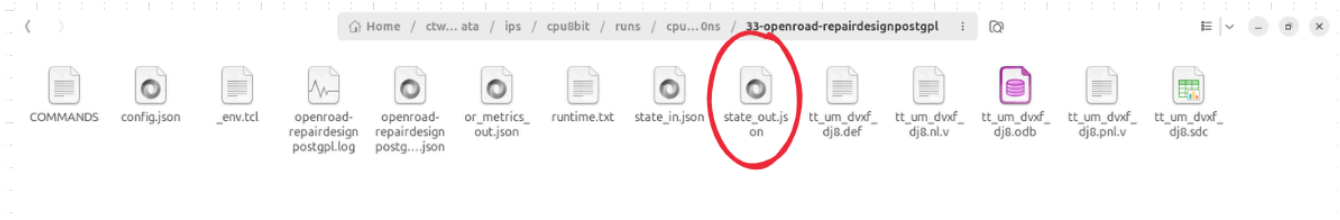
Hình 2.3: Bản đồ nhiệt và tổng hợp số cell tại bước 29 global placement



Hình 2.4: Bản đồ nhiệt và tổng hợp số cell tại bước 33 repair design post gpl

Nhận xét: Bước 33 này tools tự sửa lại netlist của Yosys và thay một vài cell có khả năng lái dòng tốt hơn để sửa max fanout (số tải tối đa mà một đầu ra của cell được lái), một vài cell sửa max cap load (điện trở và điện dung tối đa được phép trên dây tải), một vài cell sửa timing (do STA được áp dụng ở bước liền trước là 32-openroad-stamidpnr) số lượng lúc này đã tăng lên **từ 964 lên 1101** standard cell. Nó là kết quả của việc thay đổi các cell để phù hợp với các constraint vật lý. Step này cũng loại bỏ chồng lấn giữa các cell, đặt cell vào các row và site phù hợp. Dĩ nhiên logic của chip vẫn bảo toàn và không đổi.

Xem log để kiểm chứng sự thay đổi, chọn state_out.json để xem trạng thái đầu ra của bước 33 sửa các cell để thỏa ERC và timing



Hình 2.5: Thư mục chứa kết quả bước 33 chọn file state_out.js

Kết quả từ file 33-openroad-repairdesignpost/state_out.json cho thấy, bước 33 này openroad đã tự chèn vào **137 buffer cell** (đúng với lượng cell chênh lệch: **1101-964**) loại repair timing (sửa vi phạm thời gian).


```
        "design__instance__count__class:multi_input_combinational_cell": 795,
        "design__instance__area__class:multi_input_combinational_cell": 7538.48,
        "flow__warnings__count": 7,
        "flow__errors__count": 0,
        "flow__warnings__count:ODB-0220": 2,
        "flow__warnings__type_count": 2,
        "flow__warnings__count:ORD-0039": 1,
        "design__instance__count__class:fill_cell": 78,
        "design__instance__area__class:fill_cell": 292.781,
        "design__instance__count__class:tap_cell": 225,
        "design__instance__area__class:tap_cell": 281.52,
        "design__power_grid_violation__count__net:VPWR": 0,
        "design__power_grid_violation__count__net:VGND": 0,
        "design__power_grid_violation__count": 0,
        "timing__drv__floating__nets": 2,
        "timing__drv__floating__pins": 0,
        "design__instance__displacement__total": 4357.62,
        "design__instance__displacement__mean": 3.103,
        "design__instance__displacement__max": 33.412,
        "route__wirelength__estimated": 32039,
        "design__violations": 0,
        "design__instance__count__class:timing_repair_buffer": 137,
        "design__instance__area__class:timing_repair_buffer": 1278.73,
        "flow__warnings__count:RSZ-0020": 1,
        "flow__warnings__count:STA-1140": 6
    }
}
```


Hình 2.6: trạng thái chip sau khi thực hiện sửa các cell bước 33 chèn thêm 137 cell buffer

-> Lúc này số cell tăng lên đáng kể, do đó mật độ của các cell cũng tăng lên. Bạn có thể kiểm tra thông qua state_out.json của bước 29 và bước 33. Ví dụ ở bước 33, utilization là **72.9%**

```
"design__die__area": 17954.7,
"design__core__area": 16493.3,
"design__instance__count__stdcell": 1326,
"design__instance__area__stdcell": 12031.5,
"design__instance__count__macros": 0,
"design__instance__area__macros": 0,
"design__instance__count__padcells": 0,
"design__instance__area__padcells": 0,
"design__instance__count__cover": 0,
"design__instance__area__cover": 0,
"design__instance__utilization": 0.72948,
"design__instance__utilization__stdcell": 0.72948,
"design__rows": 39,
"design__rows:unithd": 39,
"design__sites": 13182,
"design__sites:unithd": 13182,
"design__instance__count__class:buffer": 1,
"design__instance__area__class:buffer": 5.0048,
"design__instance__count__class:inverter": 66,
"design__instance__area__class:inverter": 247.738,
"design__instance__count__class:sequential_cell": 102,
"design__instance__area__class:sequential_cell": 2680.07,
"design__instance__count__class:multi_input_combinational_cell": 795,
"design__instance__area__class:multi_input_combinational_cell": 7538.48,
```


Hình 2.7: Mật độ cell tăng lên đến 72.9% sau khi chèn thêm các cell buffer

Nhận xét: Nếu mật độ tăng lên tại global placement, và kiểm tra heatmap thấy đỏ toàn bộ chip (nhiều khu vực trên 100%), chứng tỏ rằng không gian trống để đặt cell hiện tại đang quá nhỏ. Nếu mật độ an toàn dưới 85%. Kiểm chứng với bước trước khi lỗi routing xảy ra, bước 38-openroad-resizertimingpostcts. Đây là bước sau khi clock tree đã được tổng hợp, và các buffer timing sửa vi phạm setup và hold time cũng được thêm vào.



Hình 2.8: Sau bước 38 tối ưu timing sau khi đã tạo clock tree synthesis số cell tổng tăng từ 1404 (b33) lên 1581 (b38)

Ngoài ra, mật độ trên 80% cũng là **báo động nguy hiểm**:



```
"design__die__bbox": "0.0 0.0 161.0 111.52",
"design__core__bbox": "2.76 2.72 158.24 108.8",
"design__io": 45,
"design__die__area": 17954.7,
"design__core__area": 16493.3,
"design__instance__count__stdcell": 1503,
"design__instance__area__stdcell": 14346.3,
"design__instance__count__macros": 0,
"design__instance__area__macros": 0,
"design__instance__count__padcells": 0,
"design__instance__area__padcells": 0,
"design__instance__count__cover": 0,
"design__instance__area__cover": 0,
"design__instance__utilization": 0.869822,
"design__instance__utilization__stdcell": 0.869822,
"design__rows": 39,
"design__rows:unithd": 39,
"design__sites": 13182,
"design__sites:unithd": 13182,
"design__instance__count__class:buffer": 1,
"design__instance__area__class:buffer": 5.0048,
"design__instance__count__class:inverter": 31,
"design__instance__area__class:inverter": 138.883,
"design__instance__count__class:sequential_cell": 102,
"design__instance__area__class:sequential_cell": 2680.07,
```

Hình 2.9: Mật độ cell là 86.98% sau bước điều chỉnh timing sau clock tree synthesis

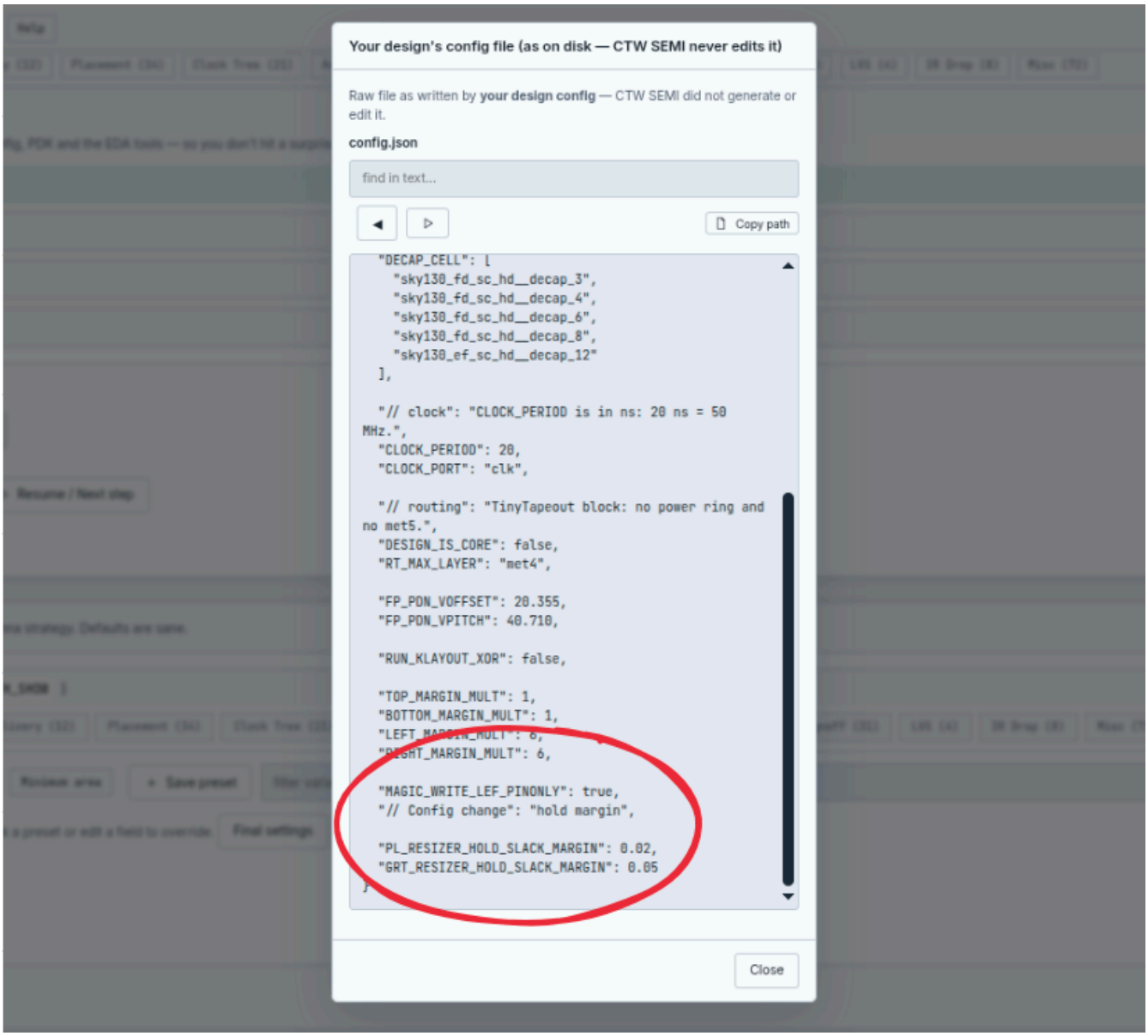
Nhận xét: Số cell chủ yếu được chèn vào là Timing Repair Buffer. Có thể kết luận ở đây, nguyên nhân là do clock hiện tại tới các cell đang quá chậm. Vấn đề timing hiện tại là vấn đề lớn nhất của chip. Ta sẽ sửa theo các cách như dưới đây. Mục tiêu là sign off với tile 1x1, không thay đổi diện tích của die và core chip.

Sửa config.json

Việc sửa timing quá mức (slack của hold vốn đã dương, nhưng được thêm 1 ngưỡng an toàn để dương hơn), có thể dẫn tới việc chèn quá nhiều buffer để sửa hold violation. Ta có thể thử giảm guardband (ngưỡng an toàn) của hold. Thêm vào cuối cùng của file config.json gốc ban đầu 2 dòng sau:

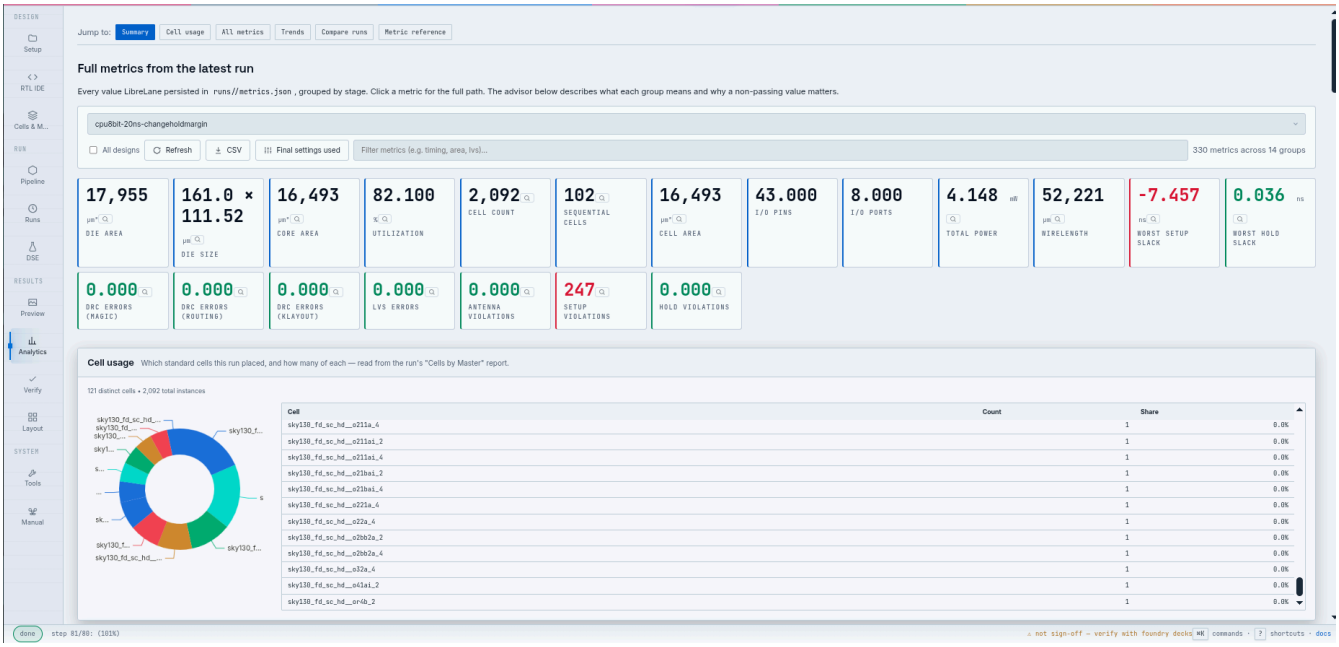
```
"// Config change": "hold margin",
 "PL_RESIZER_HOLD_SLACK_MARGIN": 0.02,
 "GRT_RESIZER_HOLD_SLACK_MARGIN": 0.05
```

Đặt tên flow là: **cpu8bit-20ns-changeholdmargin** và chạy lại flow. Trước khi chạy và recheck và kiểm tra file config.json đã cập nhật thành công hay chưa.



Hình 2.10: thêm 2 dòng config vào file config.json

Kết quả của lần chạy cpu8bit-20ns-changeholdmargin. Đã pass các bước vật lý, nhưng vi phạm setup time. Với **worst setup slack là -7.457ns**.

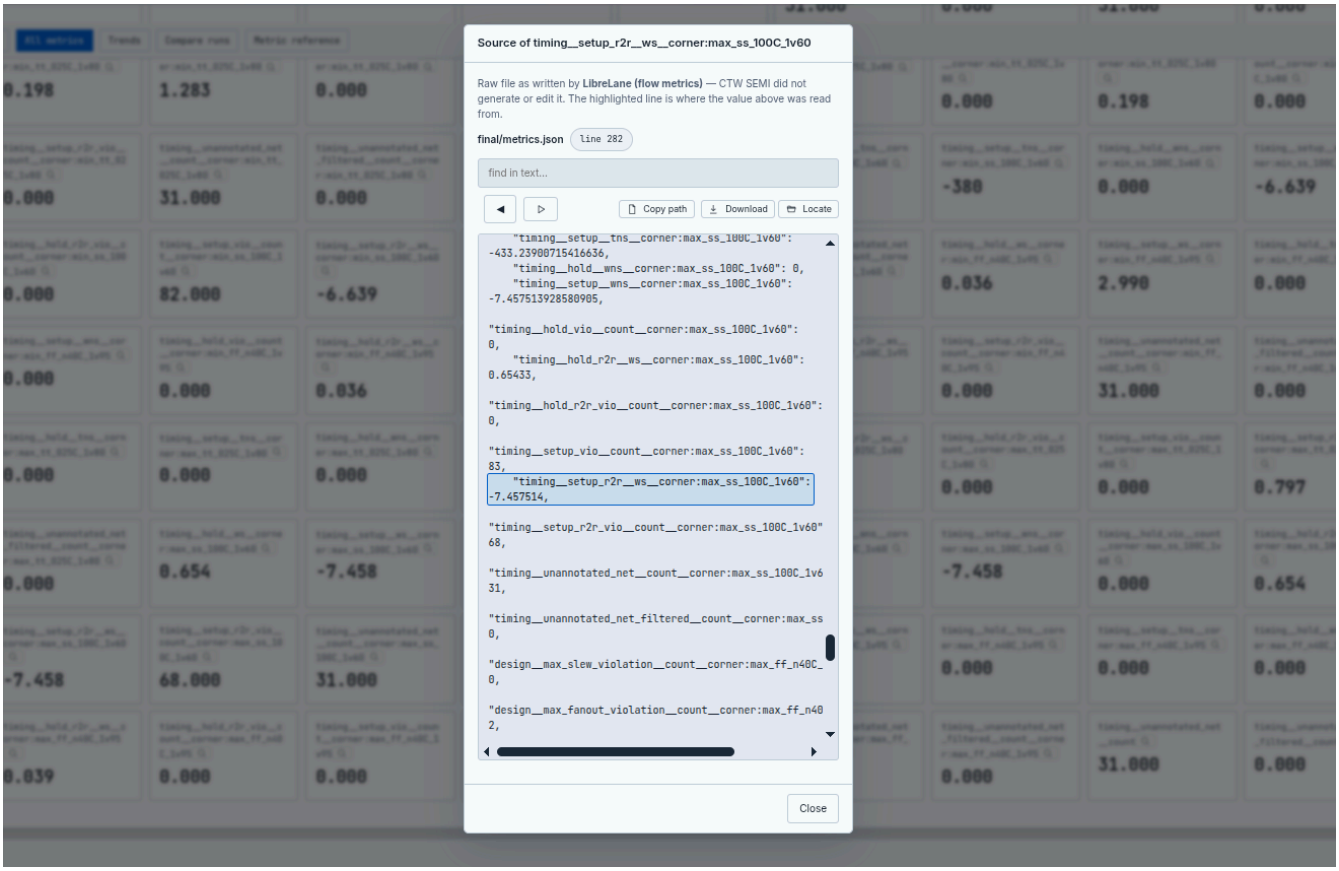


Hình 2.11: Worst setup slack là -7.457

Kiểm tra worst slack diễn ra tại corner nào:

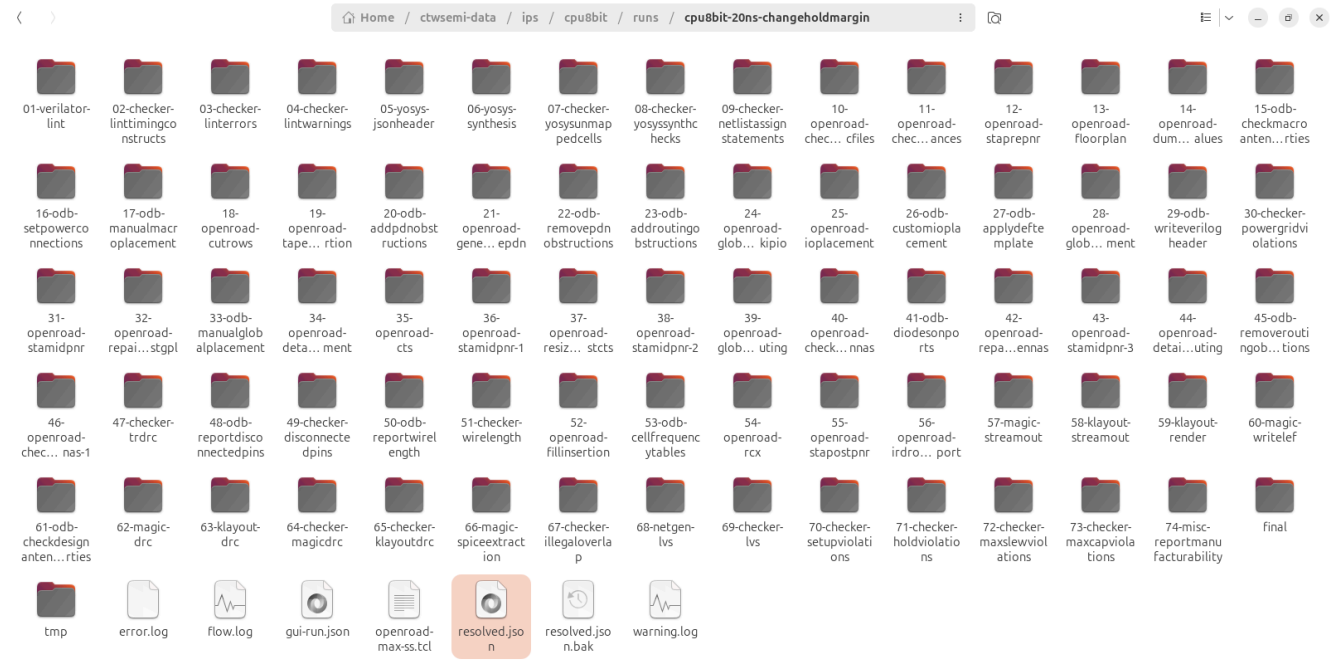


Hình 2.12: Corner max_ss_100C_1v60 là nơi có slack âm nhất.



Hình 2.13: Corner max_ss_100C_1v60 là nơi có slack âm nhất.

Mở thư mục runs tương ứng với cpu8bit-20ns-changeholdmargin, mở chọn file resolved.json.



Hình 2.14: Sửa resolved.json với text editor tại nơi lưu giữ phiên cpu8bit-20ns-changeholdmargin

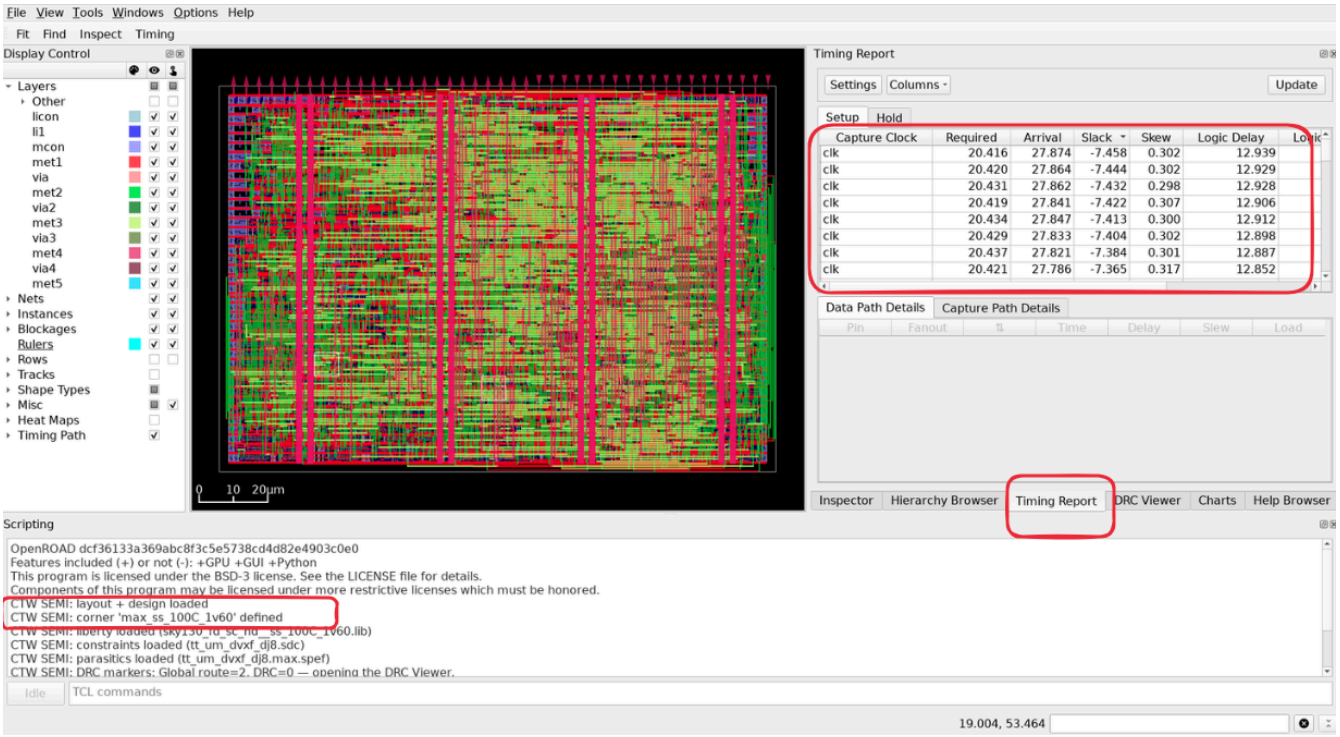
Sửa DEFAULT_CORNER thành: max_ss_100C_1v60.

```
{
    "STD_CELL_LIBRARY": "sky130_fd_sc_hd",
    "VDD_PIN": "VPWR",
    "GND_PIN": "VGND",
    "TECH_LEFS": {
        "nom_*": "/home/huyatieo/ctwsemi-data/.ciel/ciel/sky130/versions/8afc8346a57fe1ab7934ba5a6056ea8b43078e71/sky130A/libs.ref/sky130_fd_sc_hd/techlef/sky130_fd_sc_hd__nom.tlef",
        "min_*": "/home/huyatieo/ctwsemi-data/.ciel/ciel/sky130/versions/8afc8346a57fe1ab7934ba5a6056ea8b43078e71/sky130A/libs.ref/sky130_fd_sc_hd/techlef/sky130_fd_sc_hd__min.tlef",
        "max_*": "/home/huyatieo/ctwsemi-data/.ciel/ciel/sky130/versions/8afc8346a57fe1ab7934ba5a6056ea8b43078e71/sky130A/libs.ref/sky130_fd_sc_hd/techlef/sky130_fd_sc_hd__max.tlef"
    },
    "PRIMARY_GDSII_STREAMOUT_TOOL": "magic",
    "DEFAULT_MAX_TRAN": null,
    "DEFAULT_CORNER": "max_ss_100C_1v60",
    "STA_CORNERS": [
        "nom_tt_025C_1v80",
        "nom_ss_100C_1v60",
        "nom_ff_n40C_1v95",
        "min_tt_025C_1v80",
        "min_ss_100C_1v60",
        "min_ff_n40C_1v95",
        "max_tt_025C_1v80",
        "max_ss_100C_1v60",
        "max_ff_n40C_1v95"
    ],
    "RT_MIN_LAYER": "met1",
    "RT_MAX_LAYER": "met4",
    "SCL_GROUND_PINS": [
        "VGND",
        "VNB"
    ],
    "SCL_POWER_PINS": [
        "VPWR",
        "VPB"
    ],
    "TRISTATE_CELLS": [
        "sky130_fd_sc_hd__ebuf*"
    ],
    "STILL_CELLS": [
```


Hình 2.15: Sửa resolved.json thành "DEFAULT_CORNER": "max_ss_100C_1v60"

Lưu lại, sau này khi chạy Openroad GUI, tool sẽ tự động load corner này làm mặc định để phân tích thời gian tĩnh STA giữa các path.

Mở GUI của lần run cpu8bit-20ns-changeholdmargin. Xem timing của nó.



Hình 2.16: OpenROAD thông báo là corner “max_ss_100C_1v60” được xác định

Gõ các lệnh tcl sau (chức năng của các lệnh tcl cơ bản các bạn đọc trong tài liệu hướng dẫn nhé)

Một vài lệnh phổ biến cần nhớ trong bảng sau:

set	tạo biến
pwd	địa chỉ thư mục hiện tại
exec	thực thi lệnh linux, ví dụ exec ls
glob	liệt kê các file trong thư mục
$var	lấy giá trị biến
[command]	chạy command
{...}	nhóm/giữ nguyên nội dung
puts	in kết quả
foreach	lặp qua danh sách
if	rẽ nhánh
expr	tính toán
llength	đếm phần tử
lindex	lấy phần tử
string match	lọc theo tên

1. Lệnh liệt kê tên các cell theo instance (các khối có cùng chức năng, chỉ khác số thứ tự) và master (standard cell gốc suy ra instance) tương ứng của chúng.

```
set block [ord::get_db_block]


foreach inst [$block getInsts] {
  set inst_name [$inst getName]
  set master_obj [$inst getMaster]
  set master_name [$master_obj getName]
  puts "$inst_name -> $master_name"
}
// Nếu chỉ muốn hiển thị một loại cell riêng biệt, ví dụ delay cell:
```

```
set block [ord::get_db_block]


foreach inst [$block getInsts] {
 set master_obj [$inst getMaster]
 set master_name [$master_obj getName]
 if {[string match "*dly*" $master_name]} {
 puts [format "%-30s %s" [$inst getName] $master_name]
}}
```

Muốn lưu kết quả liệt kê các cell ở trên vào file output.txt (sẽ được tạo ngay tại tiny_tdc_fixsdc).

```
set block [ord::get_db_block]
set out [open "output.txt" w]  ;

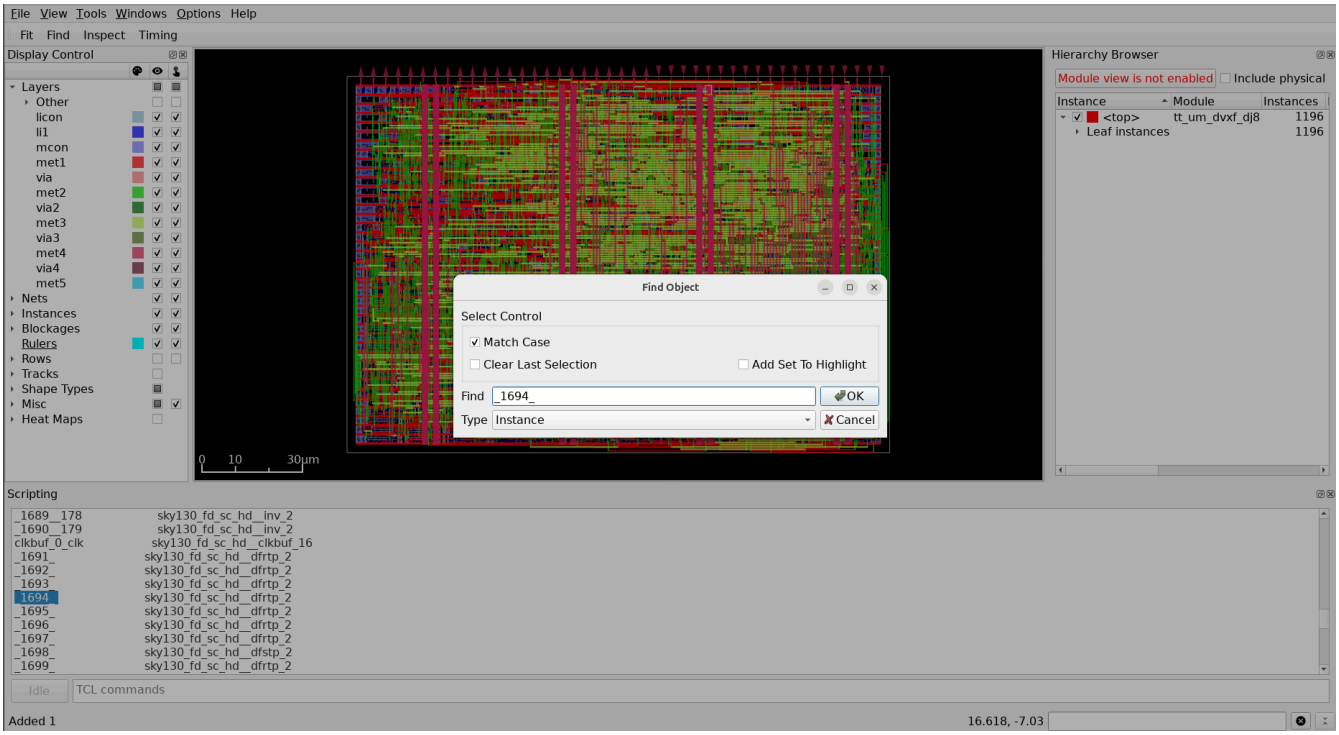
foreach inst [$block getInsts] {
  set inst_name [$inst getName]
  set master_obj [$inst getMaster]
  set master_name [$master_obj getName]
  puts $out"inst_name -> $master_name"  ;# Thêm biến $out vào trước chuỗi cần in
}

close $out  ;
```

Tên của các instance và master tương ứng



Nếu ta muốn tìm vị trí của instance, ta copy số thứ tự tương ứng. Nhấn ctrl + F để hiện cửa sổ như bên dưới:

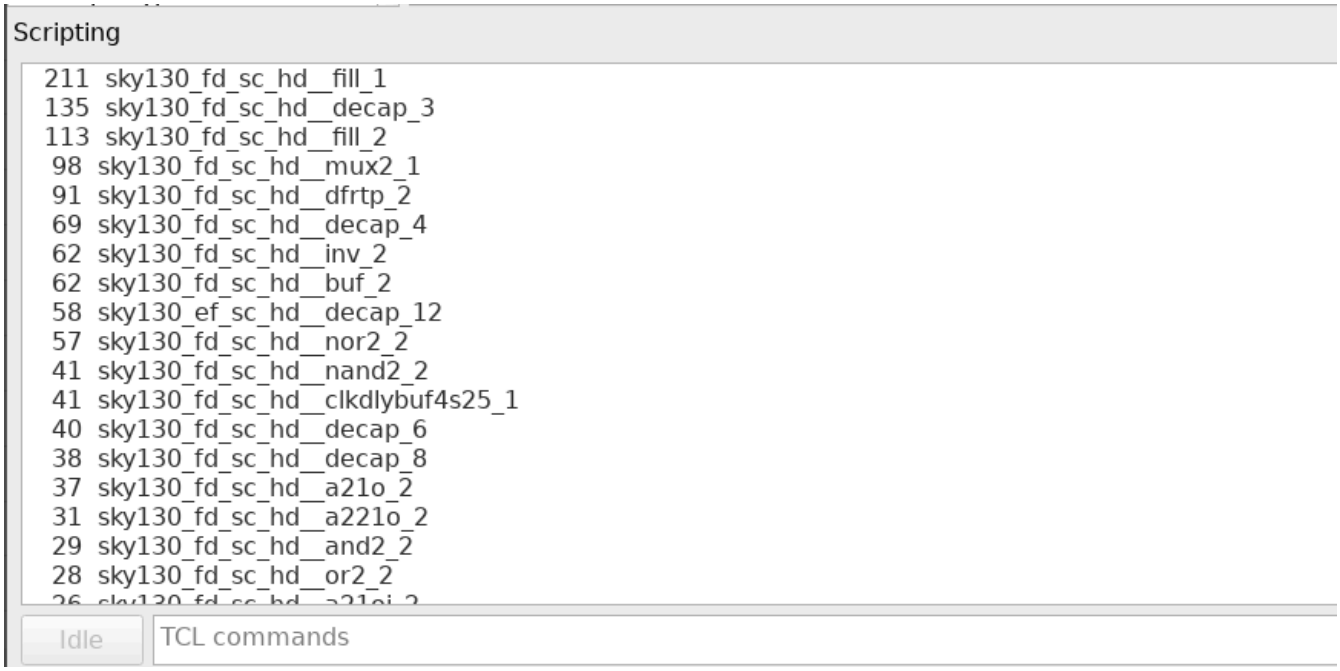


Nhấn OK, vị trí của cell sẽ sáng lên, hữu ích khi ta cần ECO (thay đổi nhỏ một vùng của thiết kế) và truy vết các path.

2. Lệnh đếm các standard cell theo từng loại, hữu ích khi quan sát sự thay đổi các cell sau khi ECO.

```
set block [ord::get_db_block]
set cell_count [dict create]

foreach inst [$block getInsts] {
  dict incr cell_count [[$inst getMaster] getName]
}
# Sắp xếp theo giá trị giảm dần
set sorted_cells [lsort -stride 2 -index 1 -integer -decreasing $cell_count]
foreach {master count} $sorted_cells {
  puts [format "%5d  %s" $count $master]
}
```



Hoặc tcl sẵn có của Openroad GUI:

```
report_cell_usage
```

3. Tìm worst path của setup:

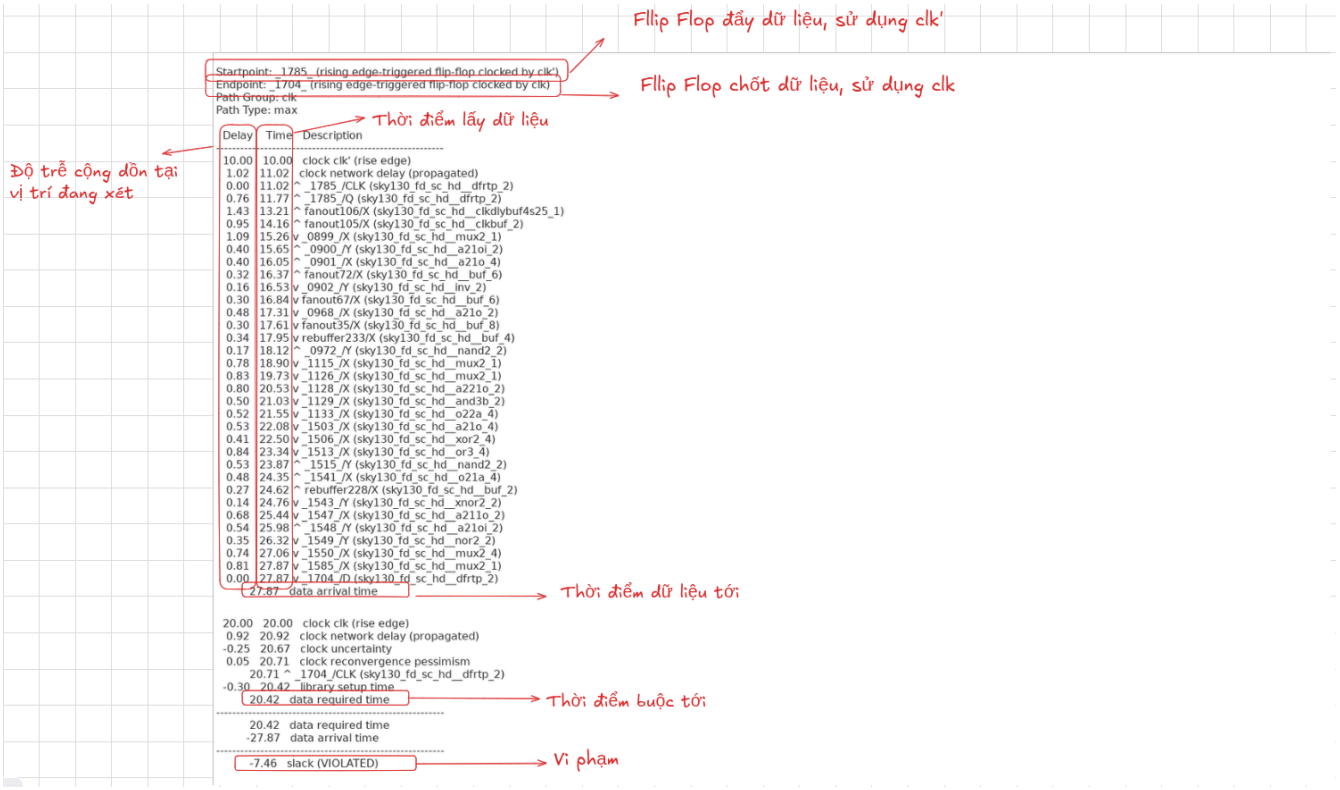
   Cần kiểm tra và đảm bảo các path được constraint (ràng buộc bởi sdc) đầy đủ, một thiết kế pass STA và timing nhưng thiếu ràng buộc thì có thể đó là pass giả.

```
check_setup -verbose
```

Kết quả sau bước check_setupnày trả về 1 nghĩa là các chân đã có ràng buộc đầy đủ, nếu trả về 0 kèm warning thì xem kĩ nếu chân này không ràng buộc theo ý định sẵn.

Chạy lệnh tìm worst path ở corner hiện tại

```
report_checks -path_delay max
# chỉ định 1 corner cụ thể
report_checks -corner nom_tt_025C_1v80 -path_delay max
```



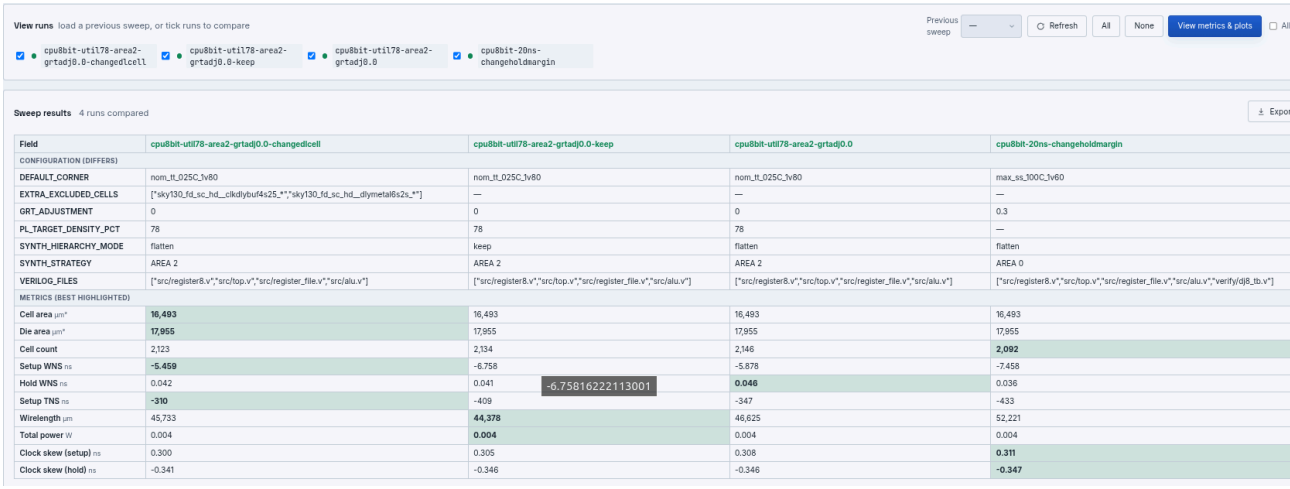
Nhận xét:

- Vì một lí do nào đó trong thiết kế, dữ liệu đẩy ra tại chân ra D của FlipFlop nguồn (*_1785_)* tại chân clk' (đây là chân clock bị đảo sau khi qua clock inverter). Do đó thay vì lẽ ra dữ liệu có thể bắt đầu đẩy ra tại 0ns, nhưng vì đẩy ra tại cạnh lên của clock đảo (tương ứng cạnh xuống của clk), do đó dữ liệu chỉ có 10ns để chạy ? Nếu nhìn vào Thời điểm lấy dữ liệu tại cột Time, ta sẽ thấy rise edge của clk' diễn ra tại 10 ns. Do đó ta có thể nói, dữ liệu chỉ có 1 nửa thời gian để chạy so với chu kì clock 20ns, đường này còn gọi là đường nửa chu kì (half cycle path). Nếu thiết kế RTL quy định dữ liệu phải chốt ở cạnh xuống, lúc này ta chỉ có thể sửa RTL thành full cycle path để dữ liệu có đủ 20ns để chạy.
- Xét tại thời điểm dữ liệu bắt đầu đi ra tại FlipFlop nguồn *_1785_/Q* là 11.77 tới khi vào chân D của FlipFlop đích là 27.87. Suy ra thời gian cần cho hệ tổ hợp là 27.87 - 11.77 = 16.1ns (lớn hơn 10ns), hệ tổ hợp còn đi qua 5 con mux lớn, do đó nếu chỉ sửa thông thường bằng ECO hoặc chạy lại toàn bộ flow RTLtoGDS, path vẫn sẽ âm.
- Có thể cải thiện một chút. Ta nhận xét thấy clkdlybuf4s25_1 khi đi qua nó mất tận 1.43 ns, clk_buf_2 là 0,95ns, đây là các cell hỗ trợ fanout, có thể gây bất lợi cho setup, có thể bắt tool dùng các cell thay thế, có thể thu hồi được khoảng 2ns cho setup và chọn những cell khác bằng cách thêm các dòng sau vào cuối config.json:

```
"EXTRA_EXCLUDED_CELLS": [
  "sky130_fd_sc_hd__clkdlybuf4s25_*",
  "sky130_fd_sc_hd__dlymetal6s2s_*",
```

```
  "sky130_fd_sc_hd__dlygate4sd3_*"
]
```

Kết quả changedlcell là tốt nhất cho thiết kế hiện tại để slack ít âm nhất với diện tích này. Nếu muốn slack dương, cần nới thêm thành 30ns (20 + 2x(5,459). Hoặc tăng thêm diện tích để thêm whitespace chèn timing buffer sửa slack.



View runs load a previous sweep, or tick runs to compare

Previous sweep — Refresh | All | None | View metrics & plots

cpu8bit-util78-area2-grtadj0.0-changedlcell | cpu8bit-util78-area2-grtadj0.0-keep | cpu8bit-util78-area2-grtadj0.0 | cpu8bit-20ns-changeholdmargin

Sweep results 4 runs compared

Field	cpu8bit-util78-area2-grtadj0.0-changedlcell	cpu8bit-util78-area2-grtadj0.0-keep	cpu8bit-util78-area2-grtadj0.0	cpu8bit-20ns-changeholdmargin
CONFIGURATION (DIFFERS)				
DEFAULT_CORNER	nom_tt_025C_1v80	nom_tt_025C_1v80	nom_tt_025C_1v80	max_ss_100C_1v60
EXTRA_EXCLUDED_CELLS	["sky130_fd_sc_hd__clkdlybuf4s25_*","sky130_fd_sc_hd__dlymetal6s2s_*"]	—	—	—
GRT_ADJUSTMENT	0	0	0	0.3
PL_TARGET_DENSITY_PCT	78	78	78	—
SYNTH_HIERARCHY_MODE	flatten	keep	flatten	flatten
SYNTH_STRATEGY	AREA 2	AREA 2	AREA 2	AREA 0
VERILOG_FILES	["src/register8.v","src/top.v","src/register_file.v","src/alu.v"]	["src/register8.v","src/top.v","src/register_file.v","src/alu.v"]	["src/register8.v","src/top.v","src/register_file.v","src/alu.v"]	["src/register8.v","src/top.v","src/register_file.v","src/alu.v","verify/dj8_tb.v"]
METRICS (BEST HIGHLIGHTED)				
Cell area µm²	**16,493**	16,493	16,493	16,493
Die area µm²	**17,955**	17,955	17,955	17,955
Cell count	2,123	2,134	2,146	**2,092**
Setup WNS ns	**-5.459**	-6.758	-5.878	-7.458
Hold WNS ns	0.042	0.041	**0.046**	0.036
Setup TNS ns	**-310**	-409	-347	-433
Wirelength µm	45,733	**44,378**	46,625	52,221
Total power W	0.004	**0.004**	0.004	0.004
Clock skew (setup) ns	0.300	0.305	0.308	**0.311**
Clock skew (hold) ns	-0.341	-0.346	-0.346	**-0.347**

-6.75816222113001

-> Khi timing vi phạm chỉ vài nanosecond, có thể tối ưu PD.

3 Thí nghiệm 2

Ví dụ với digital TDC (Timing Digital Converter dạng số).

Thiết kế RTL chi tiết đọc trong thư mục TinyTDC/sim/ có các file .drawio, mở bằng website drawio để xem kiến trúc RTL của dự án nhé .

Mở thư mục TinyTDC/src, kiểm tra config.json:



config.json

TinyTDC > src > config.json > ...

```
{
  "DESIGN_NAME": "tinytdc",
  "VERILOG_FILES": [
    "dir::tdc_delay_line_sky130.v",
    "dir::tdc_thermometer32.v",
    "dir::tinytdc.v"
  ],
  "CLOCK_PORT": "clk_i",
  "CLOCK_PERIOD": 100,
  "PNR_SDC_FILE": "dir::tinytdc_pnr.sdc",
  "SIGNOFF_SDC_FILE": "dir::tinytdc_signoff.sdc",
  "DIE_AREA": [0,0,180,115],
  "FP_SIZING": "absolute",
  "PL_TARGET_DENSITY_PCT": 78,
  "SYNTH_STRATEGY": "AREA 2",
  "TOP_MARGIN_MULT": 1,
  "BOTTOM_MARGIN_MULT": 1,
  "LEFT_MARGIN_MULT": 6,
  "RIGHT_MARGIN_MULT": 6
}
```

Nhận xét: Thiết kế gồm 3 file verilog, 2 file sdc (1 file phục vụ PnR, một file phục vụ Signoff). Diện tích giới hạn ở 180 x 115 um2 (chip có chiều rộng lớn hơn chiều dài, trông như hình chữ nhật phình ra theo chiều

ngang), khá nhỏ, do đó lựa chọn PL_TARGET_DENSITY_PCT: 78 và SYNTH_STRATEGY: AREA2, đây là kinh nghiệm sau khi thử nghiệm nhiều lần. Clock sign off mong muốn: 100ns (10MHz).

Sự khác nhau của 2 file sdc:

- PNR_SDC_FILE: Đây là file dùng để cấu hình khi đặt và đi dây, tool sẽ sửa timing cho khớp với constraint yêu cầu, thường sẽ chặt hơn sdc cho signoff (ví dụ yêu cầu chip chạy ở tần số 50MHz, tức clock 20ns, pnr_sdc sẽ thường set clock ở 18.5ns hoặc 18ns). Mục tiêu là thực tế sau khi phân tích signoff, khi đó trở và tụ kí sinh được thêm vào có thể làm chip chậm hơn đáng kể.
- SIGNOFF_SDC_FILE: Đây là file dùng để STA check sau khi thực hiện xong PnR, nó đảm bảo thiết kế thỏa constraint yêu cầu, ví dụ 20ns.

Xem file PNR_SDC_FILE, ta thấy clock yêu cầu ở 100ns, có thể set chặt hơn ở 95ns (overconstraint 5%):

```
create_clock -name clk_i -period 95.0 [get_ports {clk_i}]
set_clock_transition 0.15 [get_clocks {clk_i}]
set_clock_uncertainty 0.25 [get_clocks {clk_i}]
set sync_inputs [get_ports {arm_i rd_sel_i* start_i stop_i}]
set_input_delay -clock [get_clocks {clk_i}] -max 20.0 $sync_inputs
set_input_delay -clock [get_clocks {clk_i}] -min 0.0  $sync_inputs
set data_outputs [get_ports {rd_data_o* done_o}]
set_output_delay -clock [get_clocks {clk_i}] -max 20.0 $data_outputs
set_output_delay -clock [get_clocks {clk_i}] -min 0.0  $data_outputs



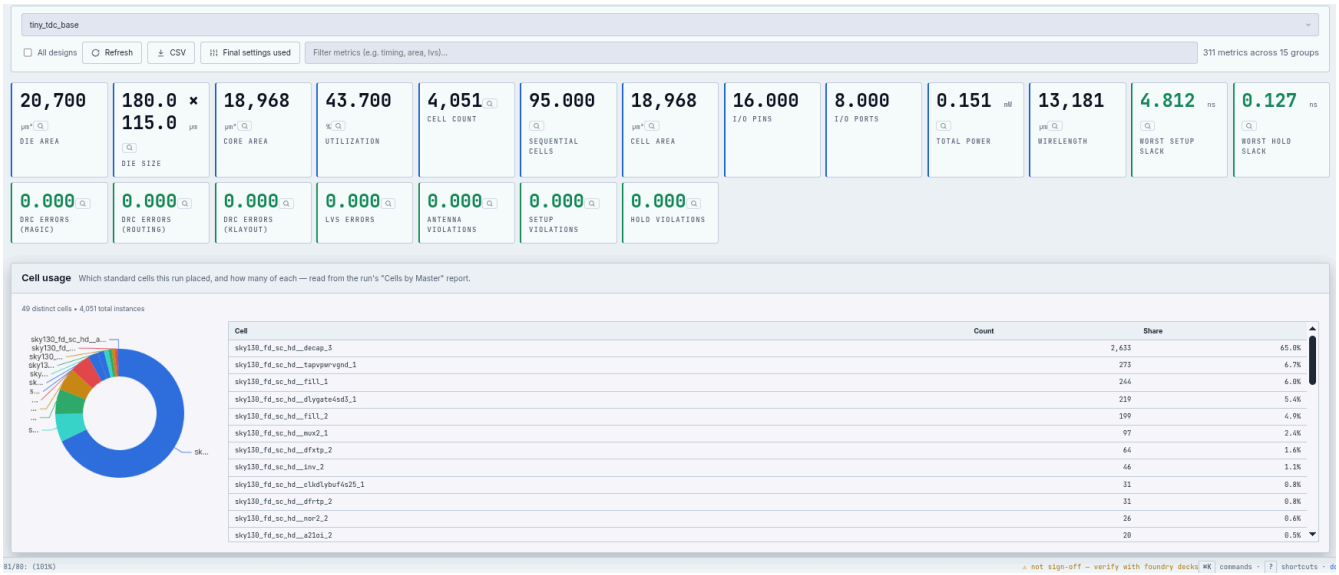
#  Asynchronous active-high reset.
set_false_path -from [get_ports {rst_i}]

set_max_transition 0.75 [current_design]
set_max_capacitance 0.20 [current_design]
set_max_fanout 10 [current_design]
```

Xem file SIGNOFF_SDC_FILE, giống hệt, chỉ thay đúng 1 dòng tạo clock:

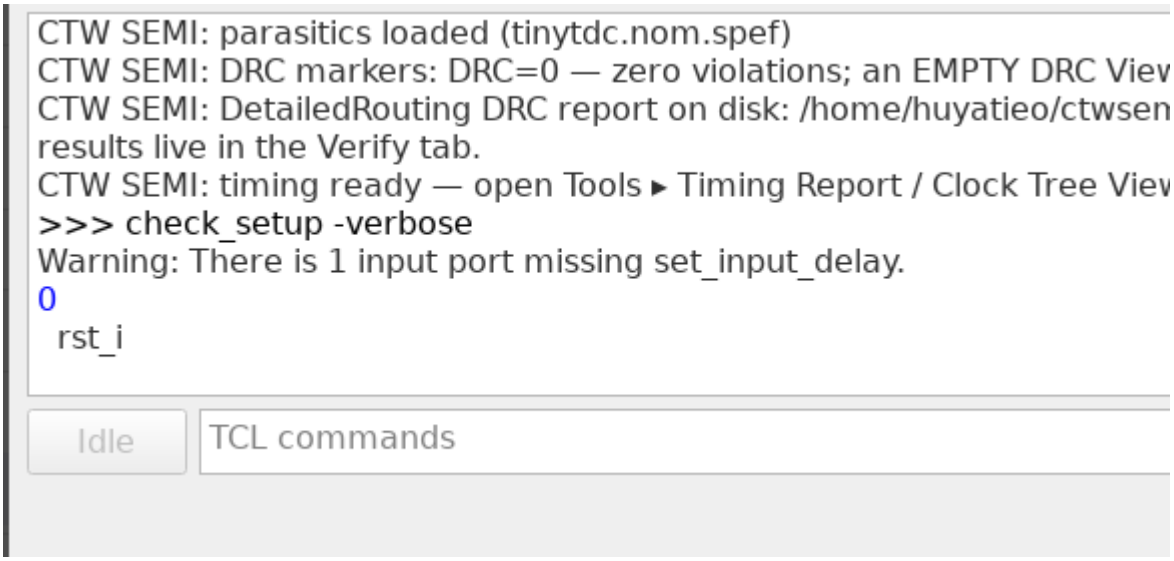
```
create_clock -name clk_i -period 100.0 [get_ports {clk_i}]
```

Chạy baseline:



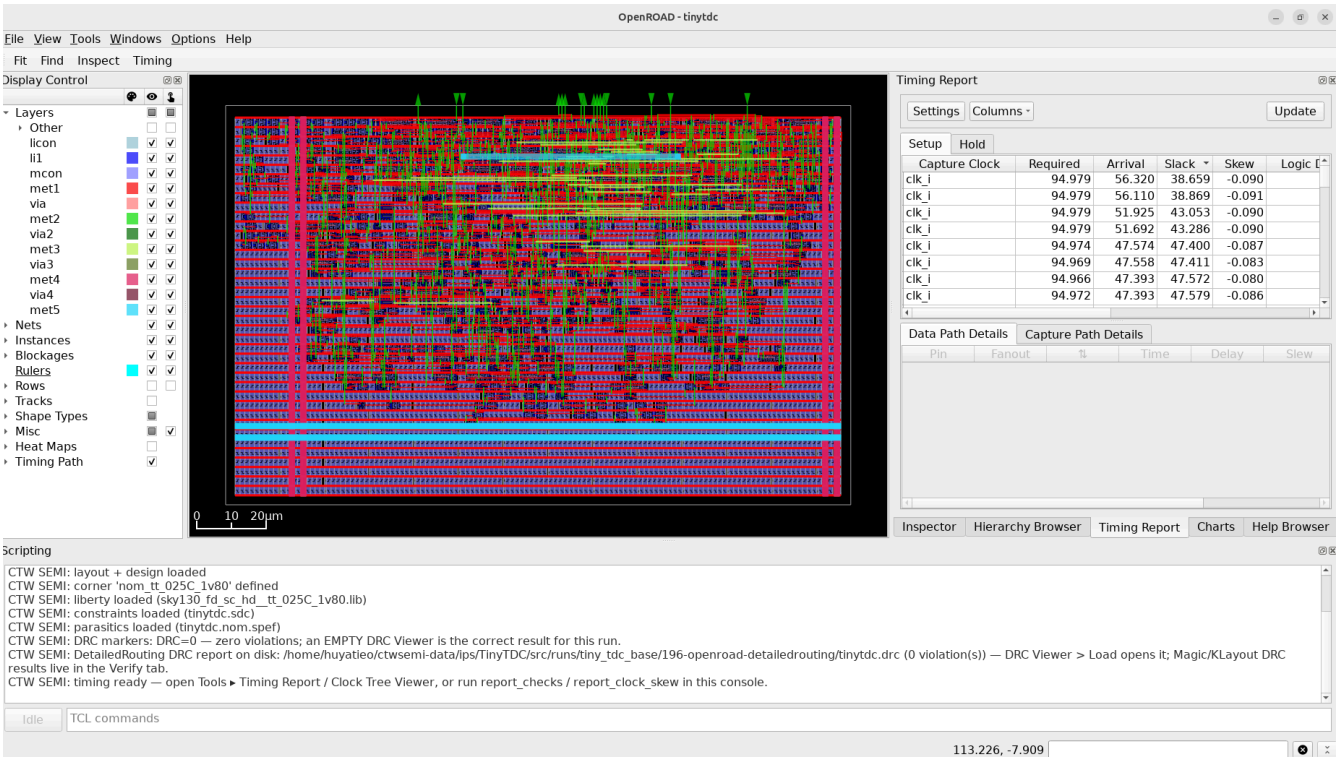
- Setup slack vẫn dương, nhưng chưa đảm bảo chip chạy đúng, vì TDC là chip đặc biệt có các phần tử trễ thực hiện chức năng, ta phải đảm bảo số lượng của chúng phải bảo toàn.
- Thiết kế thỏa area hiện tại, mật độ 43.7%.
- Hold slack dương.

Kiểm tra với Openroad GUI:



Chân rst_i là chân reset bất đồng bộ, ta cố tình set false path để chân này không tính vào khi xét STA.

Thêm cửa sổ timing_report, click update, ta thấy clock hiện tại có setup required là 94.979(tức là timing 95ns mà ta ràng buộc trong pnr sdc), cái ta cần là timing của signoff sdc (100ns).



Chạy lệnh này để openroad load đúng file sdc signoff (lưu ý đường dẫn tới file)

```
read_sdc /home/huyatieo/ctwsemi-data/ips/TinyTDC/src/tinytdc_signoff.sdc
```

Sau khi chạy lệnh này thành công, kiểm tra lại cửa sổ timing, chọn update để kiểm tra lại clock lúc này đã required ~100ns chưa:



- Mục tiêu của ta là đảm bảo 2 đường delayline cho start và stop phải bảo toàn chính xác 256 cell sky130_fd_sc_hd__dlygate4sd3_1 cho mỗi đường, cộng tổng lại số lượng sẽ là 512 cell, vì 4sd3_1 cũng dùng để sửa hold, nên tool có thể tự chèn thêm vào, tổng phải lớn 512 cell.

  Chạy lệnh này để kiểm tra số lượng cell delay trên start delay line và stop delay line:

```
set block [ord::get_db_block]

foreach line {start stop} {
    set found [dict create]

    foreach inst [$block getInsts] {
        set name [$inst getName]

        if {[string match "*u_${line}_line*g_stage*u_delay*" $name]} {
            if {[regexp {g_stage[^0-9]*([0-9]+)} $name -> index]} {
                dict set found $index 1
            }
        }
    }

    set missing {}
    for {set i 0} {$i < 256} {incr i} {
        if {![dict exists $found $i]} {
            lappend missing $i
```

```
    }
  }

  puts "===== $line delay line ====="
  puts "Found   : [dict size $found] / 256"
  puts "Missing : $missing"
}
```



Nhận xét: tool đã tự optimize một vài cell để sửa timing, vì nó cho rằng các cell này gây ảnh hưởng tới timing của hệ thống trong khi ta mong muốn giữ lại các cell này để thực hiện chức năng. Sửa cả 2 file sdc:

Từ dòng này

```
set sync_inputs [get_ports {arm_i rd_sel_i* start_i stop_i}]
```

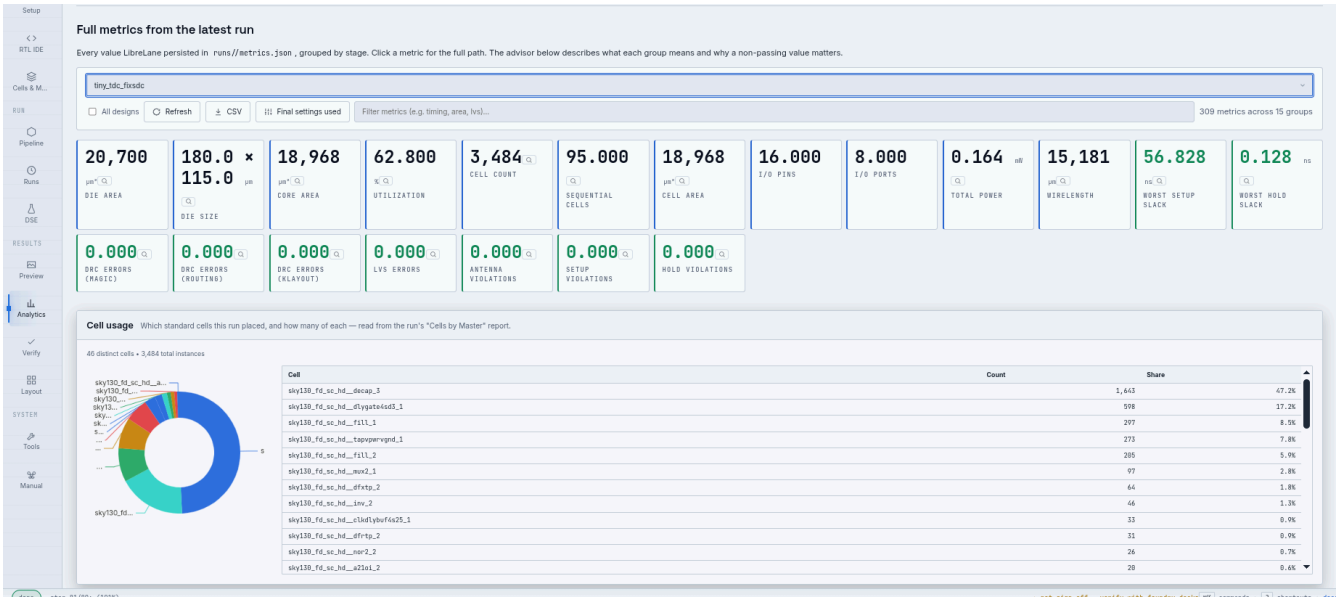
Thành:

```
set sync_inputs [get_ports {arm_i rd_sel_i*}]
```

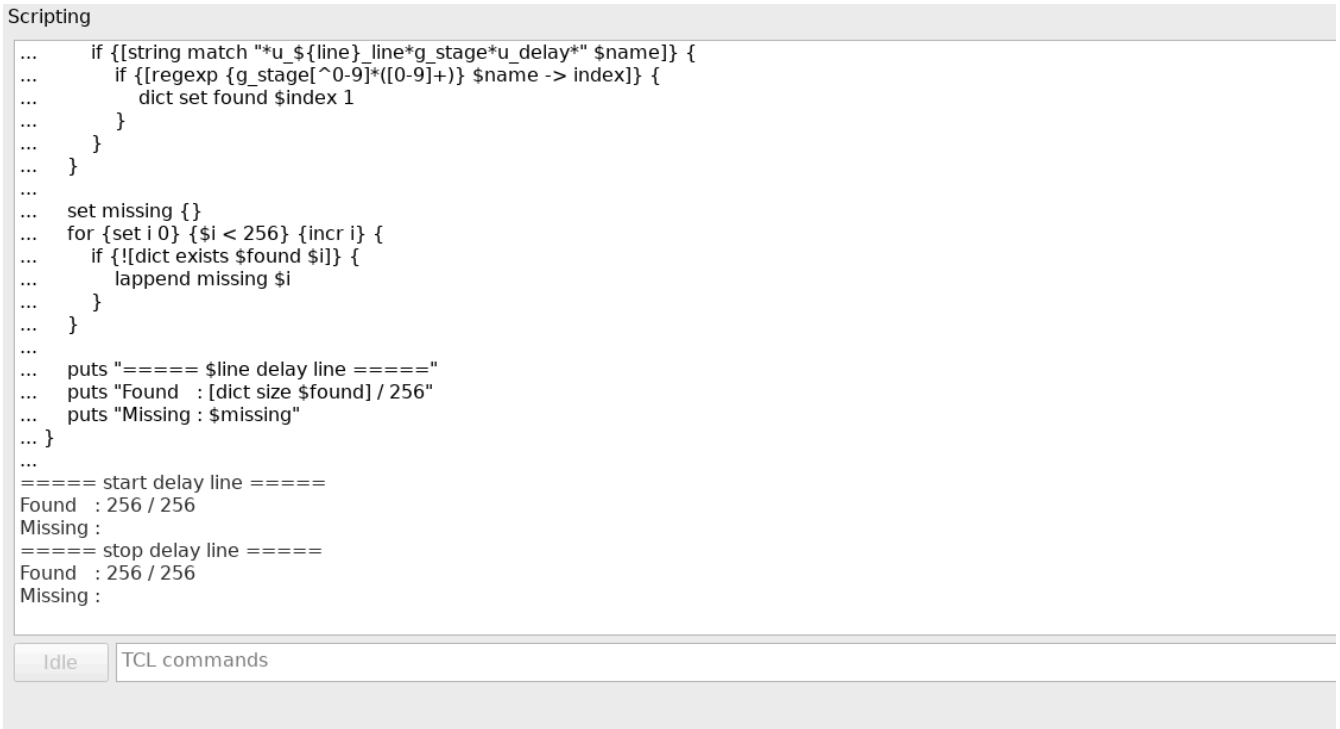
Đồng thời thêm vào:

```
#  Asynchronous input
set_false_path -from [get_ports {start_i stop_i}]
```

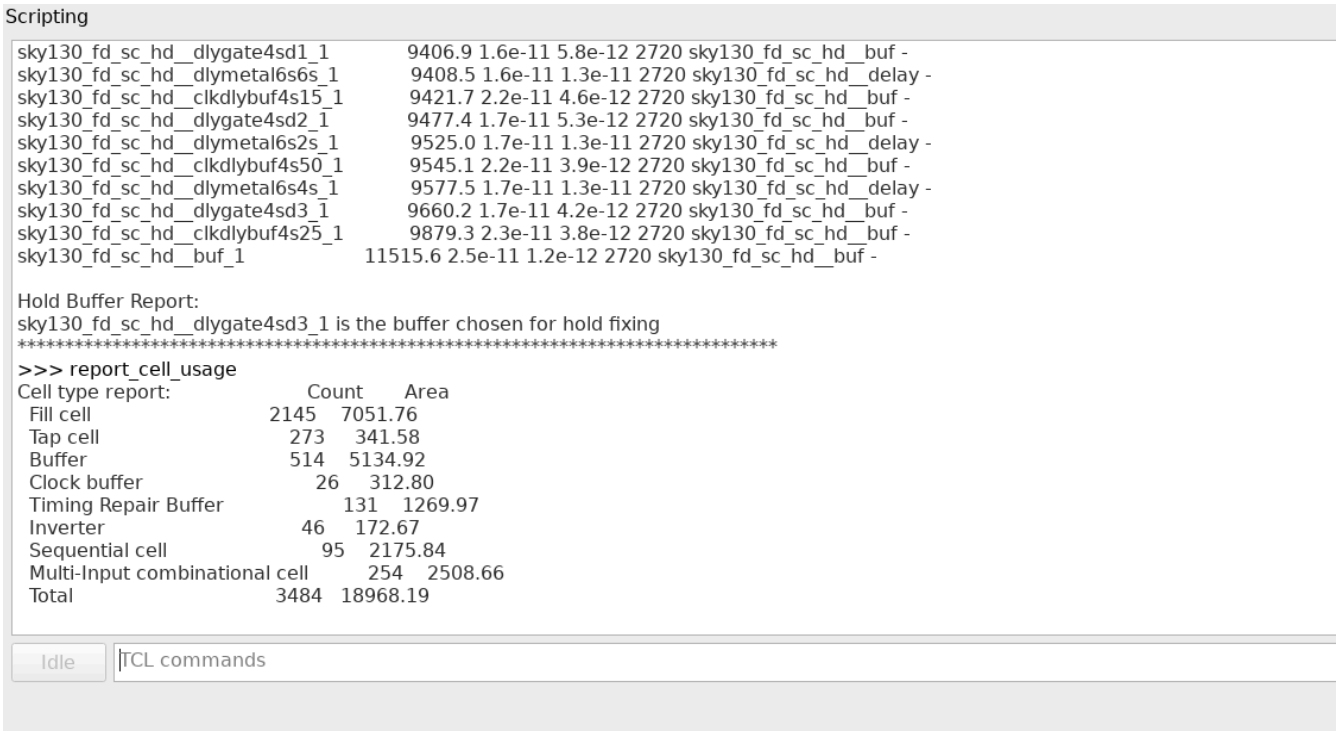
Chạy lại run mới ,đặt tên tinytdc_fixsdc, số cell và mật độ tăng lên đáng kể.



Kiểm tra lại bằng GUI:



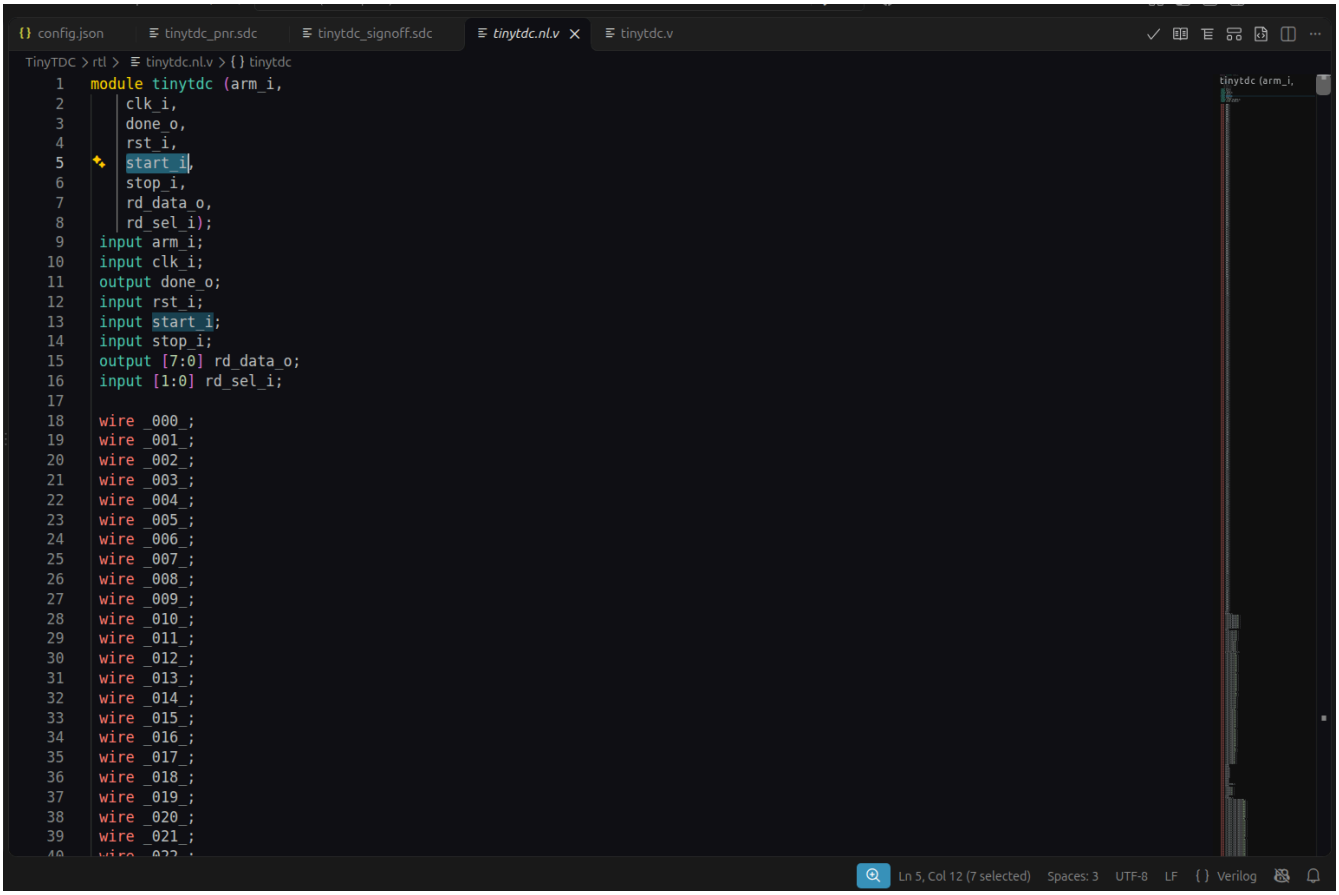
Vậy cả 256 cell đã bảo toàn, ta có thể kiểm tra:



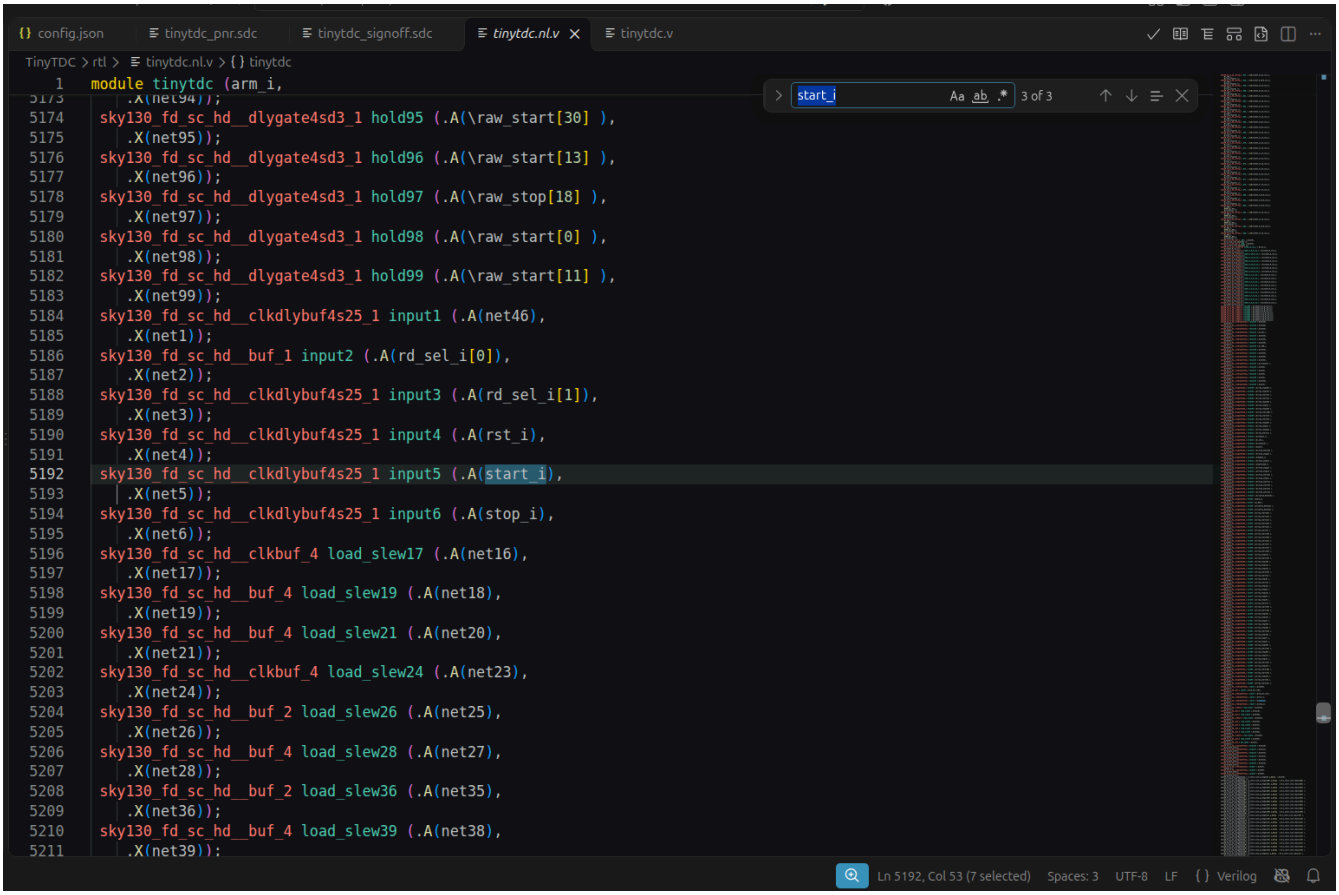
Tổng cộng có 514 cell buffer, trong đó 512 cell buffer làm chức năng của 2 delay line, 2 cell dùng để sửa hold time.

Truy vết theo netlist:

Mở file ‘tinytdc.nl.v’ được tạo tại thư mục tiny_tdc_fixsdc/final/nl/



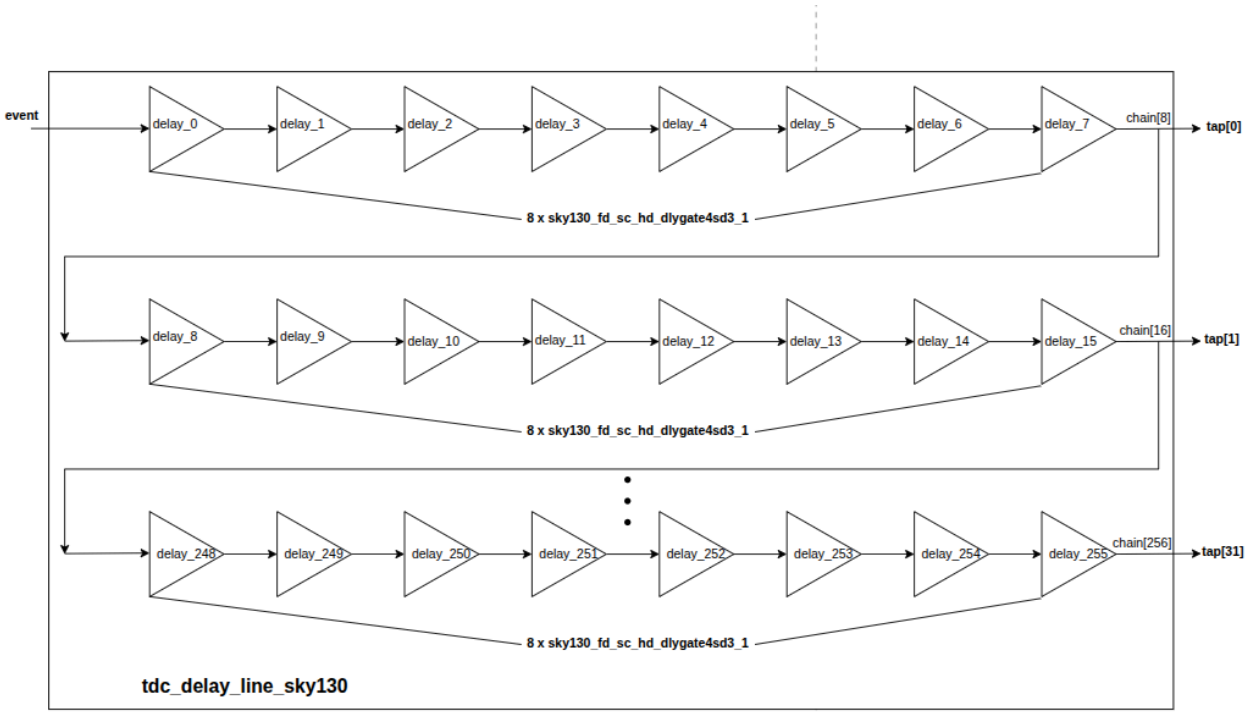
Sử dụng VisualStudio Code (VS code) để mở file, click giữ chuột trái bôi đen tín hiệu cần truy vết, ở đây là start_i, tất cả các tín hiệu cùng tên sẽ đồng loạt được bôi đen, nhìn ở thanh cuộn bên phải, sẽ thấy vị trí tương ứng start_i trong đoạn code. Hoặc đơn giản, có thể Ctrl+F để tìm tên tín hiệu:




```
1    module tinytdc (arm_i,
5174   sky130_fd_sc_hd__dlygate4sd3_1 hold95 (.A(\raw_start[30] ),
5175     .X(net95));
5176   sky130_fd_sc_hd__dlygate4sd3_1 hold96 (.A(\raw_start[13] ),
5177     .X(net96));
5178   sky130_fd_sc_hd__dlygate4sd3_1 hold97 (.A(\raw_stop[18] ),
5179     .X(net97));
5180   sky130_fd_sc_hd__dlygate4sd3_1 hold98 (.A(\raw_start[0] ),
5181     .X(net98));
5182   sky130_fd_sc_hd__dlygate4sd3_1 hold99 (.A(\raw_start[11] ),
5183     .X(net99));
5184   sky130_fd_sc_hd__clkdlybuf4s25_1 input1 (.A(net46),
5185     .X(net1));
5186   sky130_fd_sc_hd__buf_1 input2 (.A(rd_sel_i[0]),
5187     .X(net2));
5188   sky130_fd_sc_hd__clkdlybuf4s25_1 input3 (.A(rd_sel_i[1]),
5189     .X(net3));
5190   sky130_fd_sc_hd__clkdlybuf4s25_1 input4 (.A(rst_i),
5191     .X(net4));
5192   sky130_fd_sc_hd__clkdlybuf4s25_1 input5 (.A(start_i),
5193     .X(net5));
5194   sky130_fd_sc_hd__clkdlybuf4s25_1 input6 (.A(stop_i),
5195     .X(net6));
5196   sky130_fd_sc_hd__clkbuf_4 load_slew17 (.A(net16),
5197     .X(net17));
5198   sky130_fd_sc_hd__buf_4 load_slew19 (.A(net18),
5199     .X(net19));
5200   sky130_fd_sc_hd__buf_4 load_slew21 (.A(net20),
5201     .X(net21));
5202   sky130_fd_sc_hd__clkbuf_4 load_slew24 (.A(net23),
5203     .X(net24));
5204   sky130_fd_sc_hd__buf_2 load_slew26 (.A(net25),
5205     .X(net26));
5206   sky130_fd_sc_hd__buf_4 load_slew28 (.A(net27),
5207     .X(net28));
5208   sky130_fd_sc_hd__buf_2 load_slew36 (.A(net35),
5209     .X(net36));
5210   sky130_fd_sc_hd__buf_4 load_slew39 (.A(net38),
5211     .X(net39));
```

Ta thấy start_i nằm ở cell buffer tên input5, stop_i nằm ở input6.

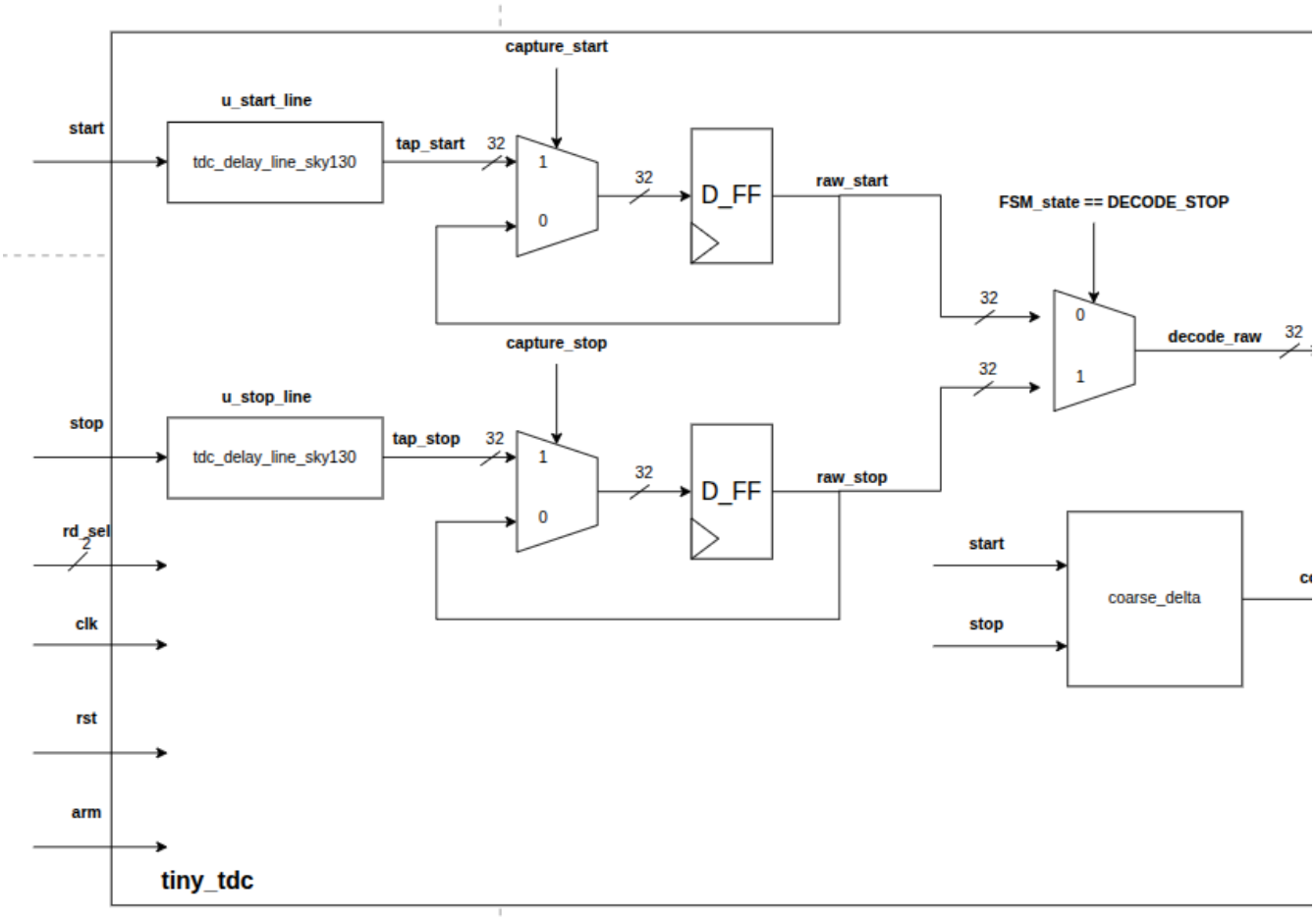
Theo sơ đồ RTL, ta có thấy vị trí của các taps tương ứng với instance chain [8], chain [15], chain [23],..



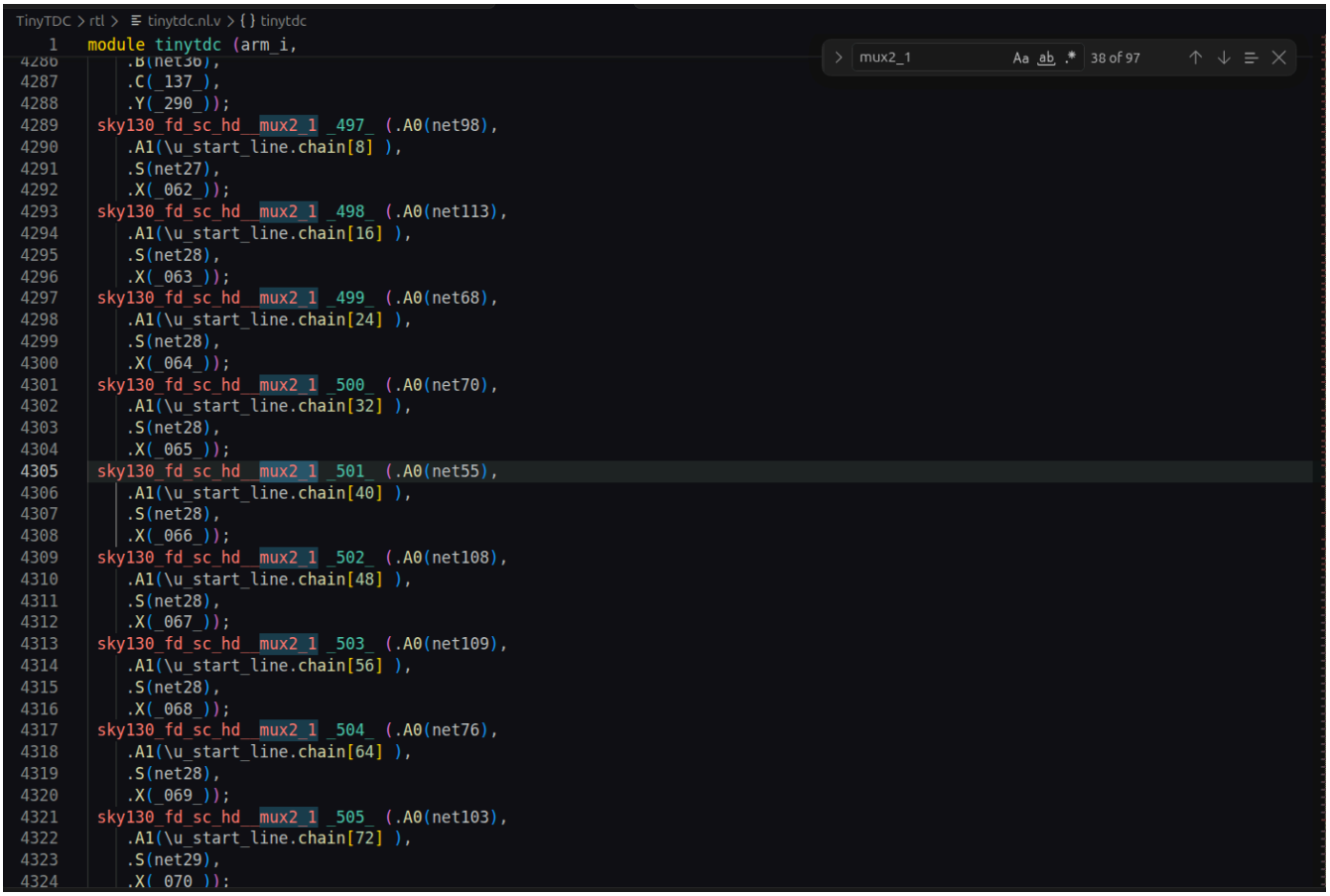
Ta tìm đúng vị trí của taps[0], của delay line start. Nằm ở ngõ ra X của instance sky130_fd_sc_hd__dlygate4sd3_1 \u_start_line.g_stage[7]. Tương tự các taps[1], taps[2], lần lượt là g_stage[15], g_stage[23],... Lưu ý vì đây là đường trễ của start, nên phải có tên u_start_line trong tên.

```
5693      .X(\u_start_line.chain[76] ));
5694    sky130_fd_sc_hd__dlygate4sd3_1 \u_start_line.g_stage[76].u_delay  (.A(\u_start_line.chain[76] ),
5695      .X(\u_start_line.chain[77] ));
5696    sky130_fd_sc_hd__dlygate4sd3_1 \u_start_line.g_stage[77].u_delay  (.A(\u_start_line.chain[77] ),
5697      .X(\u_start_line.chain[78] ));
5698    sky130_fd_sc_hd__dlygate4sd3_1 \u_start_line.g_stage[78].u_delay  (.A(\u_start_line.chain[78] ),
5699      .X(\u_start_line.chain[79] ));
5700    sky130_fd_sc_hd__dlygate4sd3_1 \u_start_line.g_stage[79].u_delay  (.A(\u_start_line.chain[79] ),
5701      .X(\u_start_line.chain[80] ));
5702    sky130_fd_sc_hd__dlygate4sd3_1 \u_start_line.g_stage[7].u_delay  (.A(\u_start_line.chain[7] ),
5703      .X(\u_start_line.chain[8] ));
5704    sky130_fd_sc_hd__dlygate4sd3_1 \u_start_line.g_stage[80].u_delay  (.A(\u_start_line.chain[80] ),
5705      .X(\u_start_line.chain[81] ));
5706    sky130_fd_sc_hd__dlygate4sd3_1 \u_start_line.g_stage[81].u_delay  (.A(\u_start_line.chain[81] ),
5707      .X(\u_start_line.chain[82] ));
5708    sky130_fd_sc_hd__dlygate4sd3_1 \u_start_line.g_stage[82].u_delay  (.A(\u_start_line.chain[82] ),
5709      .X(\u_start_line.chain[83] ));
5710    sky130_fd_sc_hd__dlygate4sd3_1 \u_start_line.g_stage[83].u_delay  (.A(\u_start_line.chain[83] ),
5711      .X(\u_start_line.chain[84] ));
5712    sky130_fd_sc_hd__dlygate4sd3_1 \u_start_line.g_stage[84].u_delay  (.A(\u_start_line.chain[84] ),
5713      .X(\u_start_line.chain[85] ));
5714    sky130_fd_sc_hd__dlygate4sd3_1 \u_start_line.g_stage[85].u_delay  (.A(\u_start_line.chain[85] ),
5715      .X(\u_start_line.chain[86] ));
5716    sky130_fd_sc_hd__dlygate4sd3_1 \u_start_line.g_stage[86].u_delay  (.A(\u_start_line.chain[86] ),
5717      .X(\u_start_line.chain[87] ));
5718    sky130_fd_sc_hd__dlygate4sd3_1 \u_start_line.g_stage[87].u_delay  (.A(\u_start_line.chain[87] ),
5719      .X(\u_start_line.chain[88] ));
```

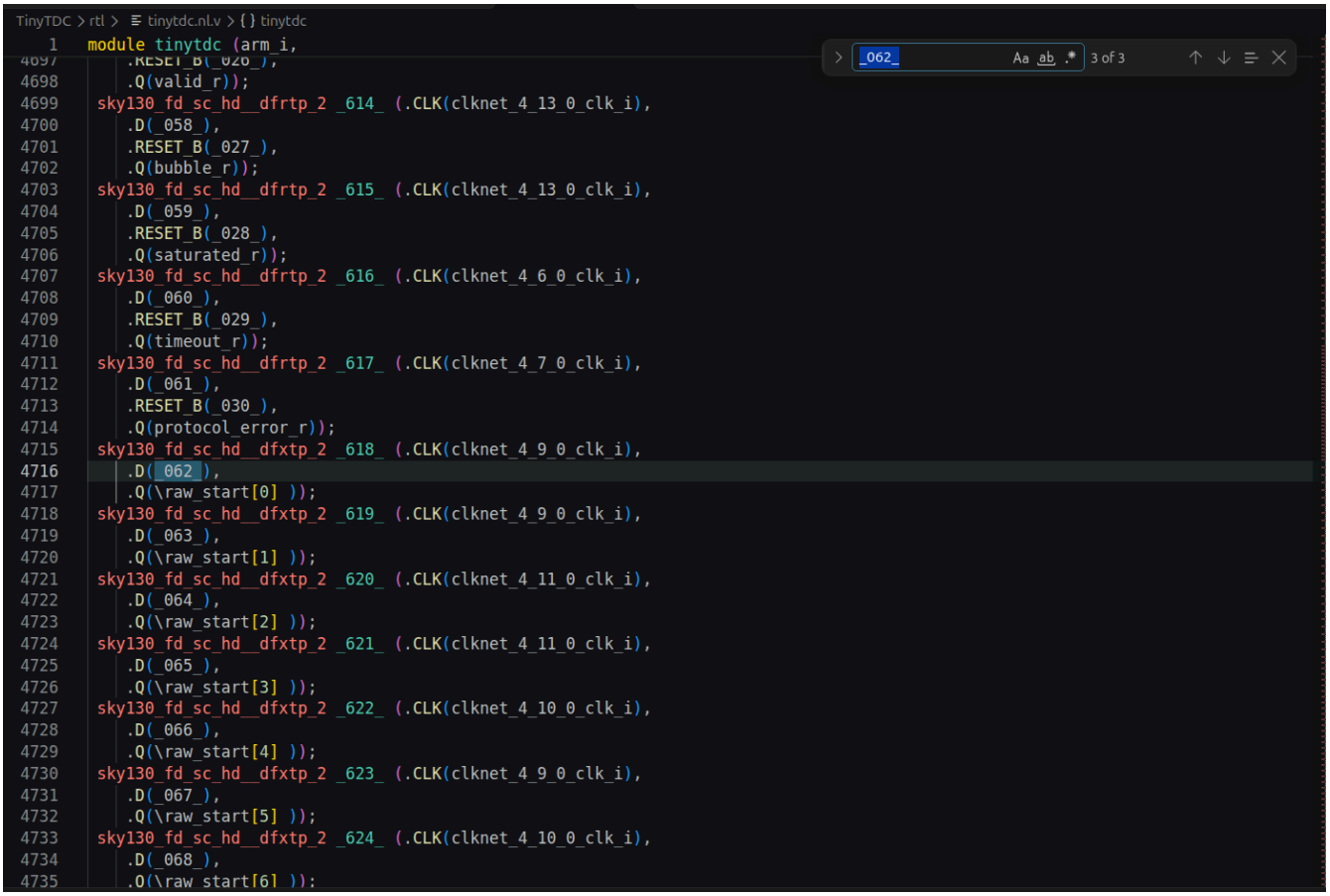
Ta truy tìm các Flops chốt cạnh lên ngõ ra của các taps. Xem kiến trúc RTL, ta thấy ngõ ra của các taps chưa đi thẳng vào chân D của Flops, mà đi qua một mux2_1, ngõ ra của mux này mới đi vào Flops. Ta sẽ lựa chọn các Flops có ngõ vào là ngõ ra của mux2_1 tương ứng.



Tìm các mux2_1 có chân đầu vào A được nối từ ngõ ra của taps[0] (chain[8]), taps[1] (chain[16]), taps[2](chain[24])...



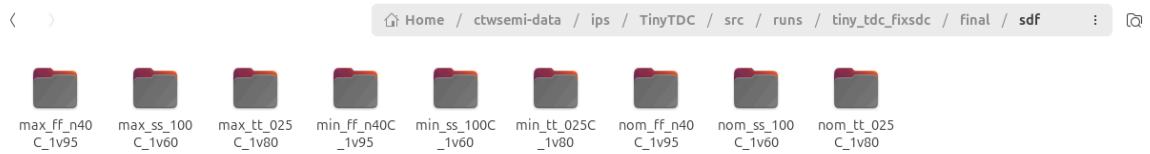
Ngõ ra của các mux2_1 này chắc chắn nối vào chân D của Flops, ví dụ với mux2_1 *_497_* có ngõ vào A là taps[0], ngõ ra X là một net *_62_*. Tìm kết nối của net này để tìm được chính xác Flops chốt dữ liệu của taps[0]. Từ đó suy ra, instance sky130_fd_sc_hd__dfxtp_2 *_618_* là Flops chốt dữ liệu của taps[0].



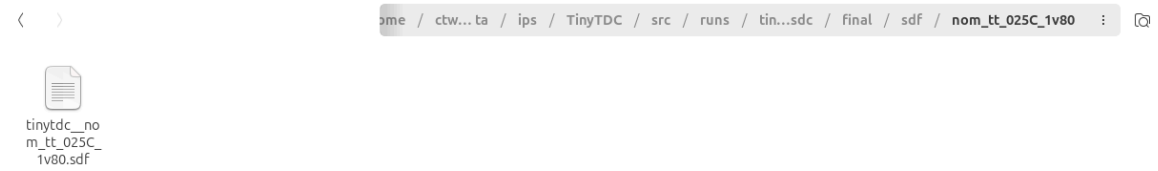
Tương tự ta có thể dễ dàng suy ra *_619_, _620_* là Flops chốt của taps[1], taps[2]...

Mô phỏng Post-layout

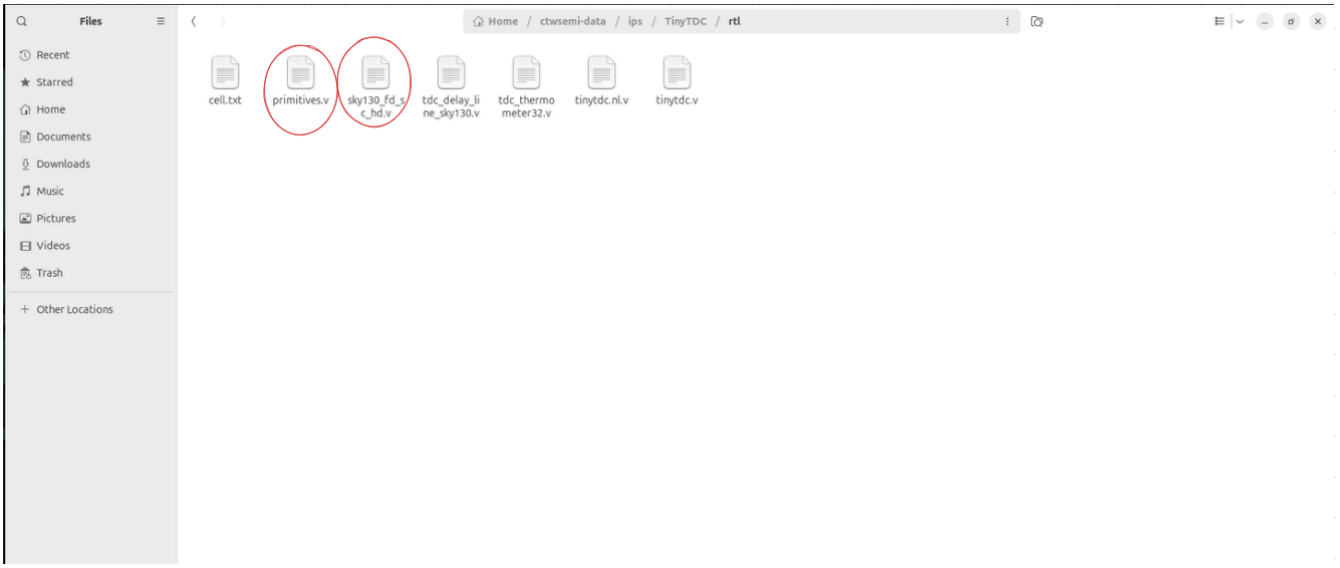
Đây là bước quan trọng nhất của thiết kế, sau khi đã PnR, các đường dây kết nối sẽ có trở và tụ kí sinh, ta cần đo chính xác thời gian trễ qua mỗi taps, của mỗi delay cell 4sd3_1, và của mỗi delay line để đảm bảo chip hoạt động ngay sau khi đã layout. Định dạng sdf (standard delay format) ghi lại các ký sinh và ước lượng độ trễ hay mô hình thời gian của từng cell sau khi PnR. Vị trí của nó nằm ở tiny_fixsdc/final/sdf/<corner>/tinytdc.sdf. Ở đây lưu ý rằng mỗi corner sẽ có các độ trễ khác nhau.



Với bài lab này, ta chỉ mô phỏng với corner nom_tt_25C_1v8, đây là điều kiện gần giống với môi trường thực tế khi chip chạy cũng như các cell danh định.



Tiếp đến, ta cần có mô hình verilog của tất các standard cell mô tả chức năng của cell theo công nghệ PDK đang dùng và thư viện synthesis, 2 files này do Skywater cấp. Đó là 2 files primitives.v và sky130_fd_sc_hd.v.



Chạy một lệnh dưới đây để cài đặt CVC (một trình mô phỏng testbench với sdf do TinyTapeout khuyến nghị, có thể dùng questasim) và kiểm tra các cài đặt cần thiết. Lưu ý phải tải thư mục TinyTDC và giải nén vào ctwsemi-data/ips/ (hoặc có thể sửa lại đường dẫn cho đúng).

```
# ============================================================
# TinyTDC-CF v0.3 — CVC Post-Layout GLS + SDF
# Tested workflow: Ubuntu 24.04 / OSS CVC 7.00b
# ============================================================

set -euo pipefail

PROJECT="$HOME/ctwsemi-data/ips/TinyTDC"

# ------------------------------------------------------------
# 1. Install dependencies and OSS CVC (one-time setup)
# ------------------------------------------------------------

sudo apt update
sudo apt install -y \
    git build-essential python3 gtkwave

if ! command -v cvc64 >/dev/null 2>&1; then
    mkdir -p "$HOME/eda-tools"
    cd "$HOME/eda-tools"

    if [ ! -d open-src-cvc ]; then
        git clone \
          https://github.com/cambridgehackers/open-src-cvc.git
    fi

    cd open-src-cvc/src
    make cvc64
    sudo make install
fi

echo "CVC location: $(command -v cvc64)"

# ------------------------------------------------------------
# 2. Check TinyTDC project files
# ------------------------------------------------------------

cd "$PROJECT"
mkdir -p sim

test -f rtl/tinytdc.nl.v || {
    echo "ERROR: Missing routed netlist"
    exit 1
}

test -f tb/tb_postlayout.v || {
    echo "ERROR: Missing post-layout testbench"
    exit 1
}
```

```
test -f tb/tinytdc_tt.sdf || {
    echo "ERROR: Missing original TT SDF"
    exit 1
}

test -f models/cvc-pdk/primitives.v || {
    echo "ERROR: Missing SKY130 primitive models"
    exit 1
}

test -d models/cvc-pdk/cells || {
    echo "ERROR: Missing split SKY130 cell models"
    exit 1
}

find models/cvc-pdk/cells -name '*.v' -print -quit |
    grep -q . || {
        echo "ERROR: SKY130 cells directory is empty"
        exit 1
    }

# ------------------------------------------------------------
# 3. Prepare SDF for OSS CVC 7.00b
#    TinyTDC-specific compatibility workaround
# ------------------------------------------------------------

cd sim

python3 - ../tb/tinytdc_tt.sdf tinytdc_tt.sdf <<'PY'
from pathlib import Path
import re
import sys

source = Path(sys.argv[1])
destination = Path(sys.argv[2])

text = source.read_text(encoding="utf-8")

# CVC 7.00b has problems annotating these output-only arcs.
# Do not change internal delay-line timing.

pattern = re.compile(
    r"(?m)^\s*\(INTERCONNECT\s+\S+\s+"
    r"rd_data_o
                                                ([1 − 7])
\s+[^\r\n]*\)\s*$"
)

removed = []

def omit(match):
    removed.append(int(match.group(1)))
    return ""
```

```
modified = pattern.sub(omit, text)

if sorted(removed) != list(range(1, 8)):
    raise SystemExit(
        f"Unexpected SDF structure; found bits {removed}"
    )

if "u_start_line" not in modified:
    raise SystemExit("START delay line missing")

if "u_stop_line" not in modified:
    raise SystemExit("STOP delay line missing")

if "IOPATH" not in modified:
    raise SystemExit("IOPATH delays missing")

destination.write_text(modified, encoding="utf-8")

print("CVC-compatible SDF generated")
print("Internal IOPATH and START/STOP paths preserved")
PY

# ------------------------------------------------------------
# 4. Post-layout gate-level simulation
# ------------------------------------------------------------

echo "Running CVC post-layout GLS..."

set +e

cvc64 \
    +interp \
    +dump2fst \
    +typdelays \
    +define+FUNCTIONAL \
    +define+UNIT_DELAY \
    ../models/cvc-pdk/primitives.v \
    -y ../models/cvc-pdk/cells \
    +libext+.v \
    ../rtl/tinytdc.nl.v \
    ../tb/tb_postlayout.v \
    2>&1 | tee cvc.log

CVC_RC=${PIPESTATUS[0]}

set -e

echo ""
echo "CVC exit code: $CVC_RC"

# ------------------------------------------------------------
# 5. Inspect simulation results
# ------------------------------------------------------------
```

```
echo ""
echo "=== Measurement results ==="

grep -E \
    "Coarse =|Fine START =|Fine STOP =|Status =" \
    cvc.log || true

echo ""
echo "=== SDF / Simulation errors ==="

grep -nE \
    'ERROR|\b[1-9][0-9]* SDF errors' \
    cvc.log || true

echo ""
echo "=== Waveform files ==="

find . -maxdepth 1 -type f \
    -name '*.vcd' -o -name '*.fst' -print

if [ "$CVC_RC" -ne 0 ]; then
    echo "FAIL: CVC returned a nonzero exit code"
    exit "$CVC_RC"
fi

echo ""
echo "Simulation command completed."
echo "Check the log for SDF errors and testbench PASS/FAIL."
```

Nếu chạy thành công tb_postlayout.v sẽ trả kết quả như sau:

```
Highest level modules:
tb_postlayout
**tinytdc_tt.sdf(9) WARN** [662] (VOLTAGE value missing in rtriple field "typ"
**tinytdc_tt.sdf(11) WARN** [662] (TEMPERATURE value missing in rtriple field "typ"
**tinytdc_tt.sdf(18) WARN** [3106] SDF (INTERCONNECT form source port arm_i is not an output or inout port of module tinytdc - works because annotated as (PORT form MIPD delay
**tinytdc_tt.sdf(19) WARN** [3106] SDF (INTERCONNECT form source port clk_i is not an output or inout port of module tinytdc - works because annotated as (PORT form MIPD delay
**tinytdc_tt.sdf(20) WARN** [3106] SDF (INTERCONNECT form source port rst_i is not an output or inout port of module tinytdc - works because annotated as (PORT form MIPD delay
**tinytdc_tt.sdf(21) WARN** [3106] SDF (INTERCONNECT form source port start_i is not an output or inout port of module tinytdc - works because annotated as (PORT form MIPD delay
**tinytdc_tt.sdf(22) WARN** [3106] SDF (INTERCONNECT form source port stop_i is not an output or inout port of module tinytdc - works because annotated as (PORT form MIPD delay
**tinytdc_tt.sdf(23) WARN** [3106] SDF (INTERCONNECT form source port rd_sel_i is not an output or inout port of module tinytdc - works because annotated as (PORT form MIPD delay
**tinytdc_tt.sdf(24) WARN** [3106] SDF (INTERCONNECT form source port rd_sel_i is not an output or inout port of module tinytdc - works because annotated as (PORT form MIPD delay

Coarse =   3
Fine START =   5
Fine STOP =  17
Status = 00000001
Halted at location **../tb/tb_postlayout.v(70) time 1114145 ps from call to $finish.
  There were 0 error(s), 51 warning(s), and 960 inform(s).

CVC exit code: 0

=== Measurement results ===
Coarse =   3
Fine START =   5
Fine STOP =  17
Status = 00000001

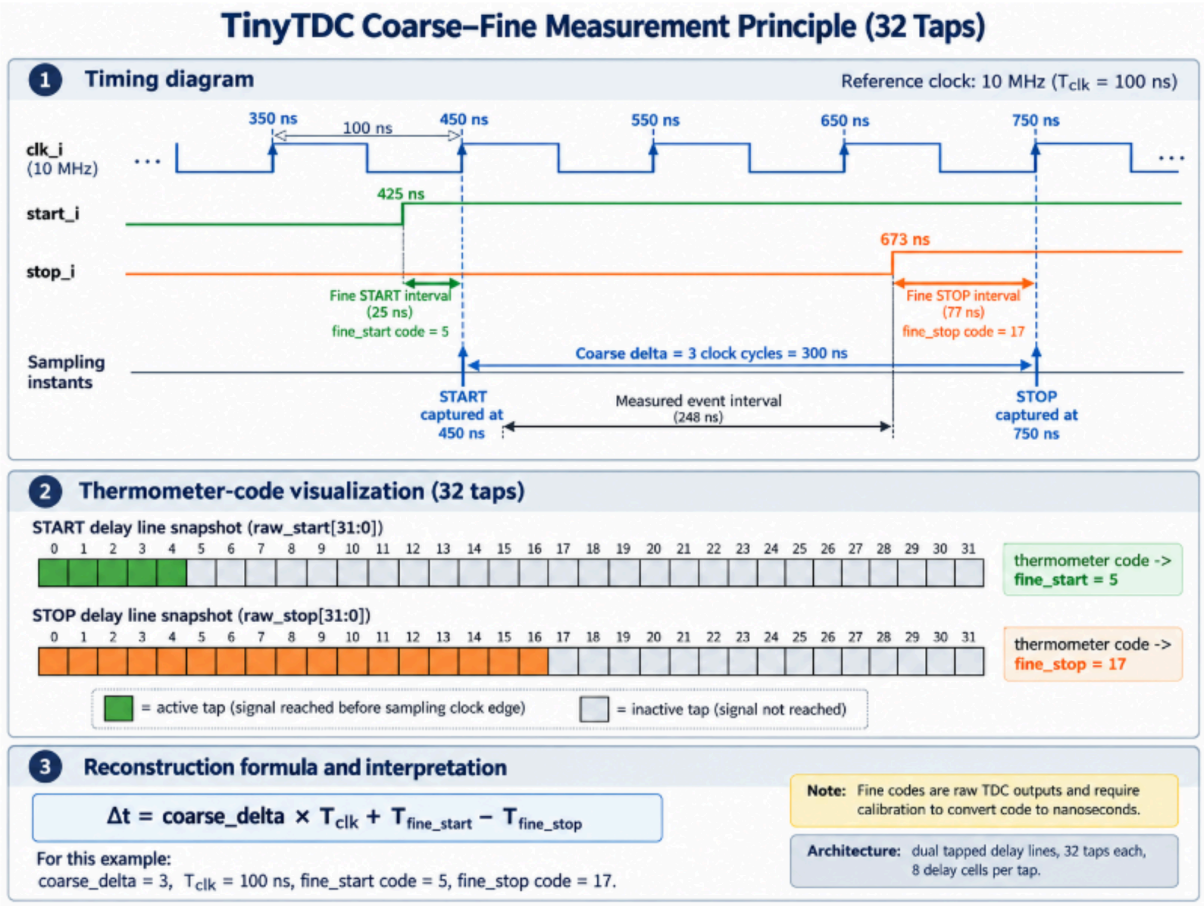
=== SDF / Simulation errors ===

=== Waveform files ===
./tinytdc_postlayout.vcd

Simulation command completed.
Check the log for SDF errors and testbench PASS/FAIL.
huyatieo@huyatieo-ThinkPad-X13-Gen-1:~/ctwsemi-data/ips/TinyTDC/sim$
```

Giải thích:

Ở ví dụ trên, testbench để chạy mô phỏng với post layout là tb_postlayout.v, đây là mô phỏng post layout GLS (mô phỏng netlist cấp cổng với độ trễ post layout). Ví dụ này đang chạy một trường hợp rất cụ thể như hình dưới đây:



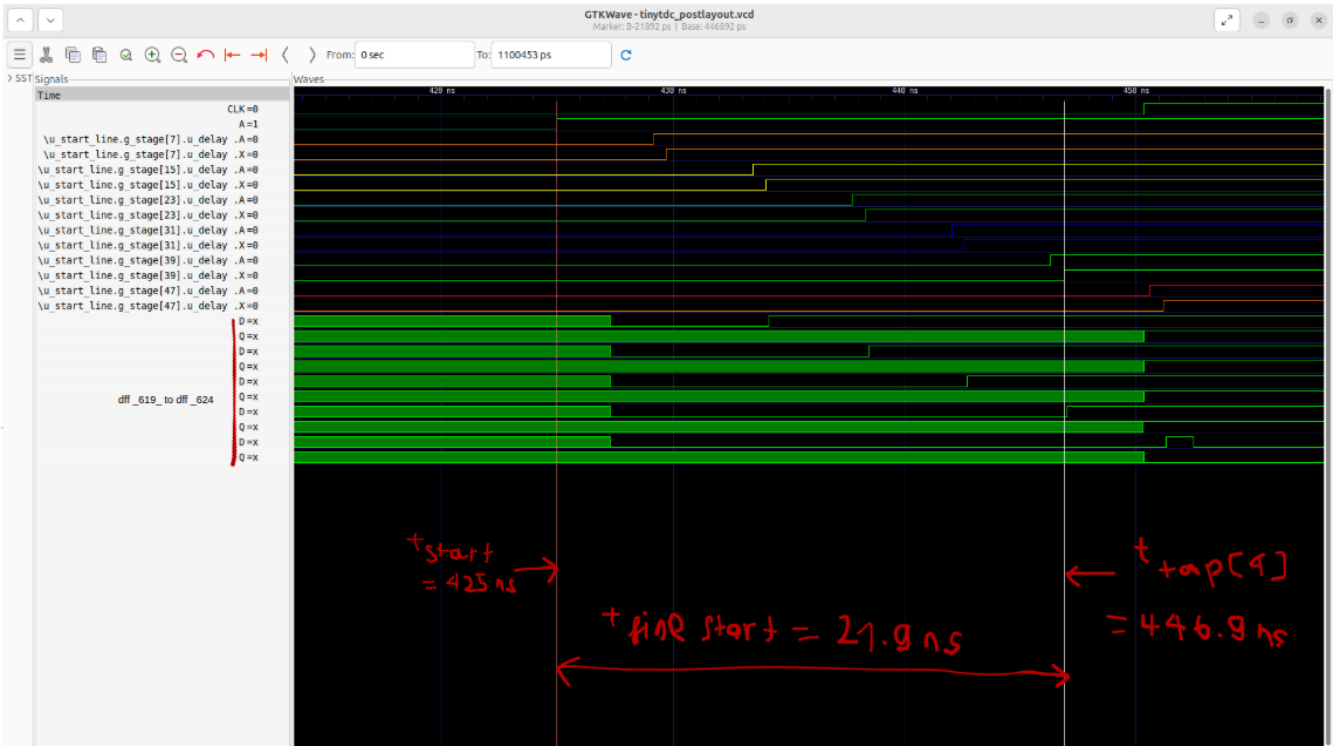
Nhận xét: Kết quả trả về Fine start = 5, Fine stop = 17, chứng tỏ rằng tại thời điểm các Flops đầu ra (*_618_,_619_,..)* của mỗi taps, dữ liệu được chốt đã trôi qua 5 taps (40 delay cell) trên đường delay start và trôi qua 17 taps (136 delay cell) trên đường delay stop, coarse = 3 nghĩa là bộ đếm thô đếm được 3 chu kì kể từ lúc Flops chốt được cạnh start và chốt được cạnh stop. Từ đó ta có thể tiến hành đo độ trễ thực tế bằng CVC để có được thời gian chính xác của delta t.

Sau khi chạy thành công CVC, mở gtkwave bằng lệnh:

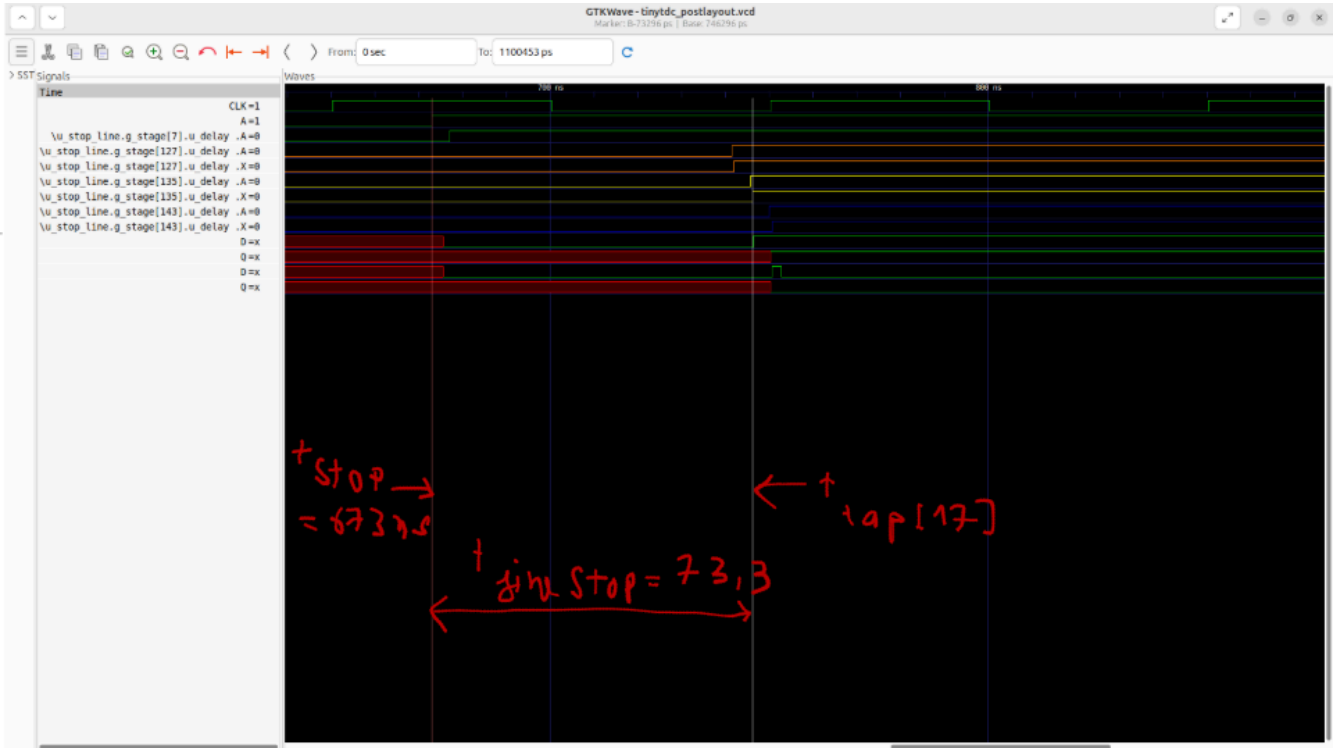
```
cd ~/ctwsemi-data/ips/TinyTDC/sim
gtkwave tinytdc_postlayout.vcd
```

Áp dụng cách truy vết theo netlist như trên đã trình bày, ta chọn chính xác các cell cần xem, Append vào như hình dưới đây. Để đo, click chuột trái vào một điểm để đặt marker ở vị trí đầu tiên (marker màu đỏ), ấn nút reload màu xanh nước biển gần ô thời gian From <> To <> để marker đỏ xuất hiện, sau đó chọn điểm thứ 2, bấm vào con lặn (ở ngay giữa chuột) để đặt marker trắng. Kết quả sẽ hiện ra Bên dưới dòng chữ GTKWave - tinytdc_postlayout.vcd.

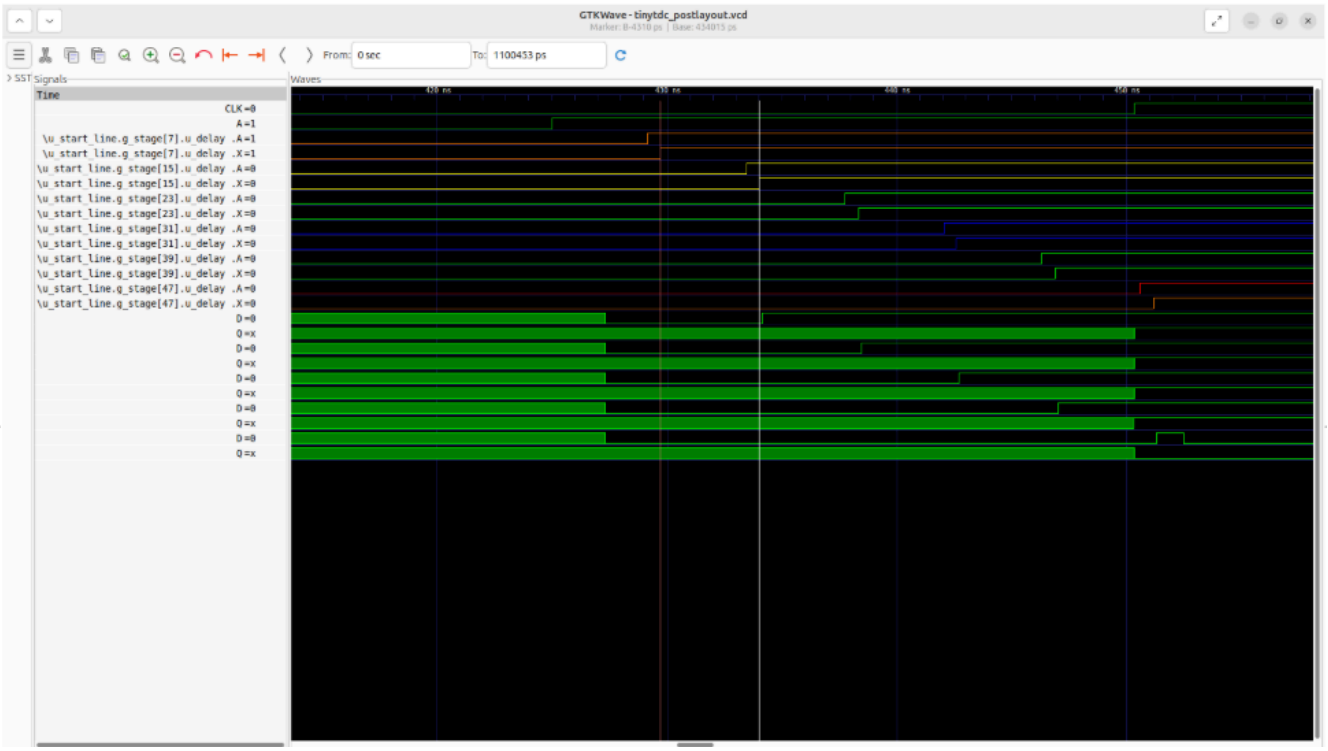
Hình: Đo Fine Start, Marker: B-21892 ps (21.9ns).



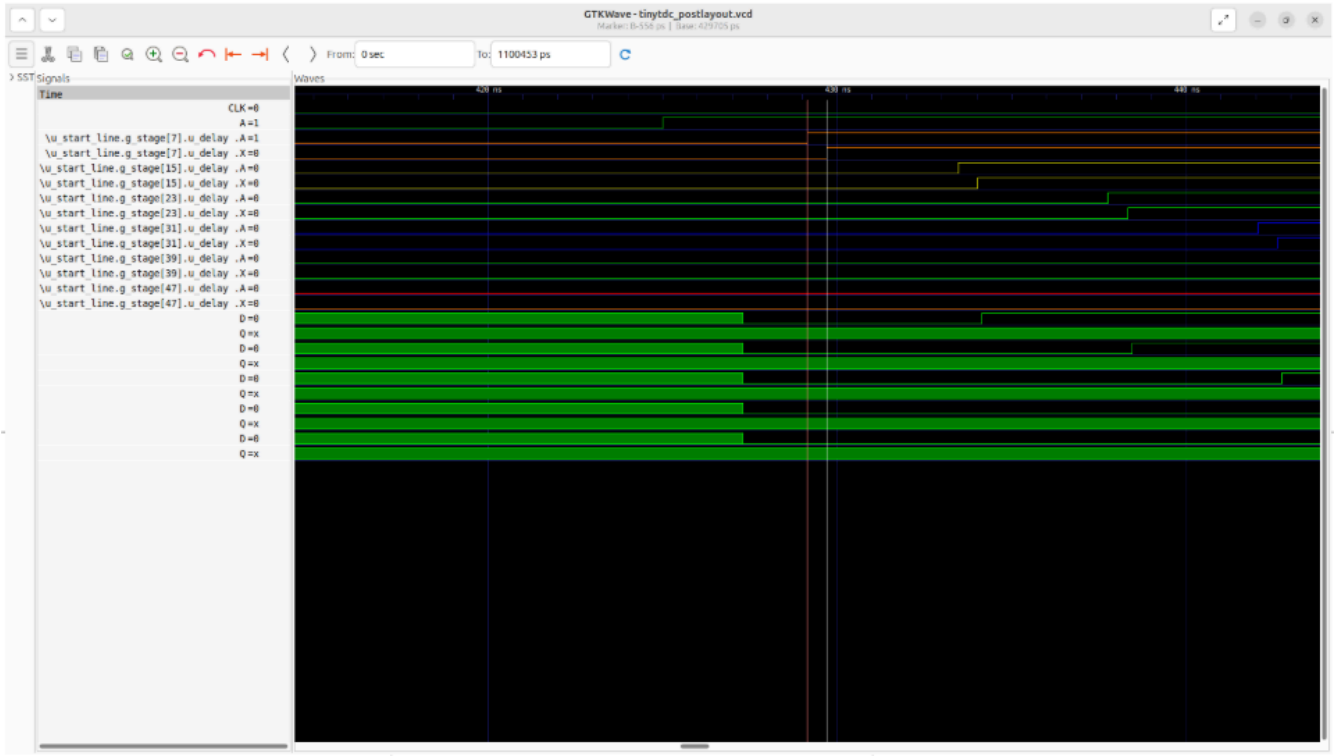
Hình: Đo Fine Stop, Marker: B-73296 ps (73.3 ns).



Hình: Đo delay của mỗi taps, ví dụ với taps[0], Marker: B-4310ps (4.3ns, chênh lệnh 0.3ps so với giả sử ban đầu 4ns):



Hình: Đo delay IOPATH (trễ vào ra) của cell delay, ở đây chỉ mới đo 1 con cell đại diện, những con cell khác trên đường trễ delay line có thể sẽ có thông số khác nhau, Marker: B-556ps (0.56ns, chênh lệch 6ps so với giả sử 0.5ns).



Hình: Đo coarse counter. Các bạn tự đo xem có đúng 300ns (3 chu kì clock không nhé? Gợi ý, xem khoảng cách Marker từ input5(start) tới input6(stop).

Từ kết quả đo thực tế ta đo được thời gian sdf vào khoảng: 3x100 + 21.9 -73.3 = 248.6ns (kết quả này có thể khác tại các góc corner khác nhau). Như có thể thấy rằng, ta chỉ thiết kế để chip TDC trả về kết quả fine start và fine stop và coarse, không trả ra một con số cụ thể như 248ns (11111000), vì khi đó ta cần một bộ hiệu chỉnh theo nhiệt độ và các góc hoạt động của chip (Thường có ở các TDC thương mại), còn khi thiết kế TDC học thuật, ta có thể dựa vào Postlayout GLS để nhờ máy tính nội suy giá trị chính xác.

Câu hỏi:

Nếu CPU 8 bit hoạt động ở clock yêu cầu là 50MHz được không ?

Nếu start và stop diễn ra cùng trong 1 chu kì ? Ví dụ start_i tại 425ns và stop_i tại 429ns thì điều gì xảy ra ? Coarse bằng bao nhiêu ? TDC lúc này có thể được khoảng thời gian gần với 429ns - 425ns là 4ns hay không ?